

Ответы на вопросы к экзамену по дисциплине «Машинное обучение и большие данные»

1. Искусственный интеллект (Artificial Intelligence, AI). Большие данные, знания. Технологии AI. Типы AI.

Искусственный интеллект (AI) — это область информатики, которая занимается созданием систем, способных выполнять задачи, требующие человеческого интеллекта. Это включает в себя обучение, рассуждение, решение проблем, восприятие и понимание языка. AI стремится имитировать и автоматизировать когнитивные функции, присущие человеку.

Большие данные — это огромные объемы информации, которые невозможно обработать традиционными методами и инструментами. Они характеризуются тремя основными характеристиками (3V): **Volume** (объем), **Velocity** (скорость обработки) и **Variety** (разнообразие форматов). **Знания** в контексте AI и больших данных — это структурированная и интерпретированная информация, которая может быть использована для принятия решений или выполнения задач.

Технологии AI включают в себя машинное обучение, глубокое обучение, обработку естественного языка, компьютерное зрение, экспертные системы и робототехнику.

Типы AI:

- **Слабый (Narrow) AI:** Специализированные системы, предназначенные для выполнения конкретных задач (например, распознавание лиц, голосовые помощники).
- **Сильный (General) AI:** Гипотетический AI, способный выполнять любую интеллектуальную задачу, которую может выполнить человек.

- **Супер-AI:** Гипотетический AI, превосходящий человеческий интеллект во всех аспектах.

2. Машинное обучение (Machine Learning, ML). Типы ML. Типы задач в ML. Примеры задач. Жизненный цикл модели ML. Схема проекта по ML.

Машинное обучение (ML) — это подраздел искусственного интеллекта, который позволяет компьютерным системам учиться на данных без явного программирования. Основная идея заключается в том, чтобы алгоритмы могли самостоятельно находить закономерности в данных и использовать их для прогнозирования или принятия решений.

Типы ML:

- **Обучение с учителем (Supervised Learning):** Алгоритмы обучаются на размеченных данных, где для каждого входного примера известен правильный ответ. Цель — предсказать выход для новых, невидимых данных.
- **Обучение без учителя (Unsupervised Learning):** Алгоритмы работают с неразмеченными данными, находя скрытые структуры или закономерности. Цель — обнаружить кластеры, ассоциации или снизить размерность данных.
- **Обучение с подкреплением (Reinforcement Learning):** Агент учится взаимодействовать со средой, получая вознаграждение за правильные действия и наказание за неправильные. Цель — максимизировать кумулятивное вознаграждение.

Типы задач в ML:

- **Классификация:** Отнесение объекта к одному из predetermined классов (например, спам/не спам, доброкачественная/злокачественная опухоль).
- **Регрессия:** Предсказание непрерывного числового значения (например, цена дома, температура воздуха).

- **Кластеризация:** Группировка похожих объектов в кластеры (например, сегментация клиентов).
- **Понижение размерности:** Уменьшение количества признаков в данных при сохранении наиболее важной информации.

Жизненный цикл модели ML:

1. **Сбор данных:** Получение релевантных данных.
2. **Предобработка данных:** Очистка, трансформация, нормализация данных.
3. **Выбор модели:** Определение подходящего алгоритма ML.
4. **Обучение модели:** Тренировка модели на обучающих данных.
5. **Оценка модели:** Проверка производительности модели на тестовых данных.
6. **Развертывание модели:** Интеграция модели в рабочую среду.
7. **Мониторинг и поддержка:** Отслеживание производительности и переобучение при необходимости.

Схема проекта по ML: Обычно включает этапы: постановка задачи, сбор и подготовка данных, выбор и обучение моделей, оценка и валидация, развертывание и мониторинг.

3. Основные понятия ML. Обучающая, валидационная и тестовая выборки. Кросс-валидация. Сравнительная характеристика методов k-fold и hold-out. Гиперпараметр, подбор гиперпараметров алгоритма ML.

Основные понятия ML:

- **Признак (Feature):** Измеряемая характеристика объекта, используемая для обучения модели.
- **Целевая переменная (Target Variable):** То, что модель должна предсказать.

- **Модель (Model):** Математическое представление, которое алгоритм ML строит на основе данных.
- **Обучение (Training):** Процесс настройки параметров модели на обучающих данных.

Обучающая, валидационная и тестовая выборки:

- **Обучающая выборка (Training Set):** Используется для обучения модели.
- **Валидационная выборка (Validation Set):** Используется для настройки гиперпараметров модели и предотвращения переобучения. Модель не обучается на этих данных напрямую.
- **Тестовая выборка (Test Set):** Используется для окончательной оценки производительности модели на новых, невидимых данных. Модель не должна видеть эти данные до финальной оценки.

Кросс-валидация (Cross-validation): Метод оценки производительности модели, при котором данные многократно делятся на обучающую и валидационную выборки. Это помогает получить более надежную оценку и снизить влияние случайного разделения данных.

Сравнительная характеристика методов k-fold и hold-out:

Характеристика	Hold-out (Простое отложенное тестирование)	k-Fold Cross-Validation (k-блочная перекрестная проверка)
Разделение	Однократное разделение на обучающую и тестовую выборки.	Многократное разделение данных на k блоков.
Использование данных	Часть данных не используется для обучения.	Все данные используются как для обучения, так и для валидации.
Надежность оценки	Менее надежная, зависит от случайности разделения.	Более надежная, усредняет результаты по k итерациям.
Вычислительная стоимость	Низкая.	Высокая, так как модель обучается k раз.
Применимость	Для больших наборов данных, где k-fold слишком затратен.	Для небольших и средних наборов данных, где нужна стабильная оценка.

Гиперпараметр: Параметр алгоритма ML, который устанавливается до начала обучения (например, скорость обучения, количество деревьев в лесу). В отличие от параметров модели, которые модель изучает из данных, гиперпараметры задаются вручную.

Подбор гиперпараметров: Процесс поиска оптимальных значений гиперпараметров для конкретной задачи. Методы включают: **Grid Search** (перебор всех комбинаций), **Random Search** (случайный перебор) и **Bayesian Optimization** (байесовская оптимизация).

4. Обучение с учителем. Проблемы ML. Решение проблемы переобучения.

Обучение с учителем (Supervised Learning) — это парадигма машинного обучения, где алгоритм учится на размеченных данных, то есть на парах «вход – правильный выход». Цель состоит в том, чтобы модель научилась отображать входные данные в выходные, чтобы она могла делать точные предсказания для новых, невидимых данных.

Проблемы ML:

- **Недостаток данных:** Нехватка достаточного количества данных для обучения модели.
- **Некачественные данные:** Зашумленные, неполные или некорректные данные.
- **Несбалансированные данные:** Неравномерное распределение классов в задачах классификации.
- **Переобучение (Overfitting):** Модель слишком хорошо запоминает обучающие данные, включая шум, и плохо обобщает на новые данные.
- **Недообучение (Underfitting):** Модель слишком проста и не может уловить основные закономерности в данных.
- **Высокая размерность данных:** Слишком много признаков, что может привести к «проклятию размерности» и усложнить обучение.

Решение проблемы переобучения:

- **Увеличение объема данных:** Чем больше данных, тем сложнее модели запомнить их все.
- **Упрощение модели:** Использование менее сложных алгоритмов или уменьшение количества параметров модели.
- **Регуляризация:** Добавление штрафа к функции потерь за сложность модели (например, L1, L2 регуляризация).
- **Кросс-валидация:** Помогает получить более надежную оценку производительности модели и выявить переобучение.
- **Ранняя остановка (Early Stopping):** Прекращение обучения, когда производительность модели на валидационной выборке начинает ухудшаться.
- **Отбор признаков (Feature Selection):** Удаление нерелевантных или избыточных признаков.
- **Ансамблевые методы:** Использование нескольких моделей для принятия решения (например, случайный лес, бустинг).

5. Разведочный анализ данных (EDA). Понятие EDA. Цель и этапы EDA. Основы описательной статистики в EDA.

Разведочный анализ данных (Exploratory Data Analysis, EDA) — это подход к анализу данных, который использует различные визуальные и статистические методы для изучения наборов данных, обобщения их основных характеристик, выявления закономерностей, обнаружения аномалий и проверки гипотез. EDA является первым и одним из важнейших шагов в любом проекте по анализу данных или машинному обучению.

Цель EDA:

- Понять структуру данных и взаимосвязи между переменными.
- Выявить пропущенные значения, выбросы и ошибки в данных.
- Проверить предположения о данных.
- Сформулировать гипотезы для дальнейшего анализа.
- Подготовить данные для моделирования.

Этапы EDA:

1. **Понимание данных:** Изучение метаданных, типов переменных, источников данных.
2. **Очистка данных:** Обработка пропущенных значений, дубликатов, некорректных записей.
3. **Одномерный анализ:** Изучение распределения каждой переменной по отдельности (гистограммы, box plots, описательная статистика).
4. **Двумерный/многомерный анализ:** Изучение взаимосвязей между парами или группами переменных (диаграммы рассеяния, корреляционные матрицы).
5. **Визуализация данных:** Создание графиков и диаграмм для наглядного представления результатов анализа.
6. **Формулирование выводов:** Обобщение полученных знаний и подготовка рекомендаций.

Основы описательной статистики в EDA:

- **Меры центральной тенденции:**
 - **Среднее (Mean):** Сумма всех значений, деленная на их количество.
 - **Медиана (Median):** Значение, которое делит отсортированный набор данных пополам.
 - **Мода (Mode):** Наиболее часто встречающееся значение в наборе данных.
- **Меры разброса (вариации):**
 - **Дисперсия (Variance):** Среднее квадратов отклонений значений от среднего.
 - **Стандартное отклонение (Standard Deviation):** Квадратный корень из дисперсии, показывает средний разброс значений относительно среднего.
 - **Размах (Range):** Разница между максимальным и минимальным значениями.
 - **Межквартильный размах (Interquartile Range, IQR):** Разница между 75-м и 25-м перцентилями, используется для определения выбросов.
- **Меры формы распределения:**
 - **Асимметрия (Skewness):** Мера несимметричности распределения.
 - **Экссесс (Kurtosis):** Мера «остроты» пика распределения.

6. Типы данных в EDA. Предобработка данных. Инструменты визуализации EDA. Визуализация зависимостей признаков в наборе данных.

Типы данных в EDA:

- **Качественные (номинальные/категориальные):** Представляют категории или группы (например, цвет глаз, пол). Не имеют числового значения.
- **Порядковые:** Категории, которые имеют естественный порядок (например, размер одежды: S, M, L).
- **Количественные (числовые):** Представляют измеряемые величины.

- **Дискретные:** Могут принимать только целые значения (например, количество детей).
- **Непрерывные:** Могут принимать любое значение в заданном диапазоне (например, рост, вес).

Предобработка данных: Процесс подготовки сырых данных для анализа и моделирования. Включает:

- **Очистка данных:**
 - **Обработка пропущенных значений:** Удаление строк/столбцов, заполнение средним, медианой, модой или с помощью ML-моделей.
 - **Обработка выбросов:** Удаление, трансформация или замена аномальных значений.
 - **Удаление дубликатов:** Идентификация и удаление повторяющихся записей.
- **Трансформация данных:**
 - **Масштабирование/Нормализация:** Приведение числовых признаков к одному диапазону (например, `MinMaxScaler` , `StandardScaler`).
 - **Кодирование категориальных признаков:** Преобразование категорий в числовой формат (например, `One-Hot Encoding` , `Label Encoding`).
 - **Создание новых признаков (Feature Engineering):** Извлечение или создание дополнительных признаков из существующих для улучшения качества модели.

Инструменты визуализации EDA:

- **Python-библиотеки:** `Matplotlib` , `Seaborn` , `Plotly` , `Pandas` (встроенные функции визуализации).
- **R-пакеты:** `ggplot2` .
- **BI-инструменты:** `Tableau` , `Power BI` .

Визуализация зависимостей признаков в наборе данных:

- **Для двух количественных признаков:**
 - **Диаграмма рассеяния (Scatter Plot):** Показывает взаимосвязь между двумя переменными.

- **Линейный график (Line Plot):** Для временных рядов или упорядоченных данных.
- **Для количественного и категориального признаков:**
 - **Ящичковая диаграмма (Box Plot):** Показывает распределение количественного признака для каждой категории.
 - **Скрипичная диаграмма (Violin Plot):** Аналогично Box Plot, но показывает плотность распределения.
- **Для двух категориальных признаков:**
 - **Столбчатая диаграмма (Bar Plot):** Показывает частоту или долю каждой категории.
 - **Тепловая карта (Heatmap):** Для визуализации таблиц сопряженности.
- **Для множества признаков:**
 - **Корреляционная матрица (Correlation Matrix) с тепловой картой:** Показывает попарные корреляции между всеми числовыми признаками.
 - **Pair Plot (Seaborn):** Строит диаграммы рассеяния для всех пар признаков и гистограммы для отдельных признаков.

7. Задача регрессии. Линейная модель. Метод наименьших квадратов. Функция потерь. Метрики оценки регрессии.

Задача регрессии: В машинном обучении это тип задачи, где целью является предсказание непрерывного числового значения (целевой переменной) на основе одного или нескольких входных признаков. Примеры: предсказание цены дома, температуры, уровня продаж.

Линейная модель (Linear Model): Простейшая модель регрессии, которая предполагает линейную зависимость между входными признаками и целевой переменной. Математически это выражается как: $y = b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_n \cdot x_n$, где y — предсказываемое значение, x — признаки, а b — коэффициенты (параметры модели), которые нужно найти.

Метод наименьших квадратов (Ordinary Least Squares, OLS): Наиболее распространенный метод для обучения линейной регрессии. Его цель — найти такие значения коэффициентов b , которые минимизируют сумму квадратов разностей между фактическими значениями целевой переменной и значениями, предсказанными моделью. Это обеспечивает наилучшее приближение прямой (или гиперплоскости) к данным.

Функция потерь (Loss Function) / Функция стоимости (Cost Function): Мера того, насколько хорошо модель выполняет свою задачу. В регрессии часто используется **Среднеквадратичная ошибка (Mean Squared Error, MSE)**, которая рассчитывается как среднее арифметическое квадратов разностей между фактическими и предсказанными значениями. Чем меньше значение функции потерь, тем лучше модель.

Метрики оценки регрессии: Используются для количественной оценки производительности регрессионных моделей:

- **Среднеквадратичная ошибка (MSE):** $MSE = (1/n) * \sum (y_i - \hat{y}_i)^2$. Чем меньше, тем лучше. Чувствительна к выбросам.
- **Корень из среднеквадратичной ошибки (RMSE):** $RMSE = \sqrt{MSE}$. Имеет ту же размерность, что и целевая переменная, что облегчает интерпретацию.
- **Средняя абсолютная ошибка (MAE):** $MAE = (1/n) * \sum |y_i - \hat{y}_i|$. Менее чувствительна к выбросам, чем MSE.
- **Коэффициент детерминации (R-squared, R^2):** Показывает долю дисперсии целевой переменной, объясняемую моделью. Значения от 0 до 1 (иногда может быть отрицательным). Чем ближе к 1, тем лучше модель. $R^2 = 1 - (MSE_{model} / MSE_{baseline})$, где $MSE_{baseline}$ — MSE модели, которая всегда предсказывает среднее значение целевой переменной.

8. Задача регрессии. Многомерная линейная регрессия, проблема мультиколлинеарности.

Регуляризованная регрессия. Полиномиальная регрессия.

Многомерная линейная регрессия (Multiple Linear Regression): Расширение простой линейной регрессии, где целевая переменная предсказывается на основе нескольких входных признаков. Уравнение остается линейным относительно коэффициентов, но учитывает вклад каждого признака.

Проблема мультиколлинеарности (Multicollinearity): Возникает, когда два или более входных признака в модели линейной регрессии сильно коррелируют друг с другом. Это может привести к нестабильности оценок коэффициентов, затруднить их интерпретацию и снизить обобщающую способность модели. Решения: удаление одного из коррелирующих признаков, объединение признаков, использование регуляризации.

Регуляризованная регрессия (Regularized Regression): Методы, которые добавляют штрафной член к функции потерь линейной регрессии для уменьшения сложности модели и предотвращения переобучения. Это помогает уменьшить величину коэффициентов или обнулить некоторые из них.

- **Lasso-регрессия (L1-регуляризация):** Добавляет штраф, пропорциональный абсолютным значениям коэффициентов. Может обнулять некоторые коэффициенты, выполняя отбор признаков.
- **Ridge-регрессия (L2-регуляризация):** Добавляет штраф, пропорциональный квадратам значений коэффициентов. Уменьшает коэффициенты, но редко обнуляет их.
- **Elastic Net:** Комбинация L1 и L2 регуляризации.

Полиномиальная регрессия (Polynomial Regression): Тип регрессии, в котором зависимость между независимой переменной x и зависимой переменной y моделируется как полином n -й степени. Это позволяет моделировать нелинейные зависимости, используя линейную модель (линейную по коэффициентам, но не по признакам). Например, $y = b_0 + b_1 \cdot x + b_2 \cdot x^2$.

9. Задача классификации. Алгоритмы классификации в ML. Проблема дисбаланса

классов и ее решение. Методы сэмплирования.

Метрики оценки классификации. Ошибки классификации.

Задача классификации: В машинном обучении это тип задачи, где целью является отнесение объекта к одному из predetermined дискретных классов или категорий. Примеры: определение спама в электронной почте, диагностика заболевания (болен/здоров), распознавание рукописных цифр.

Алгоритмы классификации в ML:

- **Линейные модели:** Логистическая регрессия, SVM (линейный).
- **Древовидные модели:** Деревья решений, случайный лес, градиентный бустинг.
- **Метрические модели:** k-ближайших соседей (kNN).
- **Байесовские классификаторы:** Наивный байесовский классификатор.
- **Нейронные сети:** Многослойный перцептрон.

Проблема дисбаланса классов (Class Imbalance): Возникает, когда количество примеров одного класса (мажоритарного) значительно превышает количество примеров другого класса (миноритарного). Это может привести к тому, что модель будет плохо предсказывать миноритарный класс, так как она будет оптимизироваться под мажоритарный.

Решение проблемы дисбаланса классов:

- **Методы сэмплирования:**
 - **Oversampling (передискретизация):** Увеличение количества примеров миноритарного класса (например, путем дублирования или генерации синтетических данных, как в SMOTE).
 - **Undersampling (недодискретизация):** Уменьшение количества примеров мажоритарного класса.
- **Изменение функции потерь:** Присвоение большего веса ошибкам на миноритарном классе.
- **Использование других метрик:** Вместо точности (accuracy) использовать F1-меру, Precision, Recall, ROC-AUC.

- **Ансамблевые методы:** Специализированные алгоритмы, такие как `BalancedBaggingClassifier`.

Метрики оценки классификации:

- **Матрица ошибок (Confusion Matrix):** Таблица, показывающая количество правильных и неправильных предсказаний для каждого класса.
 - **True Positive (TP):** Правильно предсказанный положительный класс.
 - **True Negative (TN):** Правильно предсказанный отрицательный класс.
 - **False Positive (FP):** Неправильно предсказанный положительный класс (ошибка I рода).
 - **False Negative (FN):** Неправильно предсказанный отрицательный класс (ошибка II рода).
- **Accuracy (Точность):** $(TP + TN) / (TP + TN + FP + FN)$. Доля правильных предсказаний. Плохо работает при дисбалансе классов.
- **Precision (Точность):** $TP / (TP + FP)$. Доля правильно предсказанных положительных среди всех, кого модель назвала положительными.
- **Recall (Полнота) / Sensitivity:** $TP / (TP + FN)$. Доля правильно предсказанных положительных среди всех фактических положительных.
- **F1-мера:** Гармоническое среднее Precision и Recall. $2 * (Precision * Recall) / (Precision + Recall)$. Хорошо подходит для несбалансированных данных.
- **ROC-AUC (Receiver Operating Characteristic - Area Under the Curve):** Площадь под кривой ROC, которая показывает зависимость TPR (Recall) от FPR ($FP / (FP + TN)$) при различных порогах классификации. Чем ближе к 1, тем лучше модель.

10. Линейная модель классификации:

логистическая регрессия. Различия между

линейной регрессией и логистической регрессией.

Вероятностная модель: наивный байесовский классификатор.

Линейная модель классификации: Логистическая регрессия (Logistic Regression): Несмотря на название «регрессия», это алгоритм для решения задач бинарной классификации. Он использует сигмоидную (логистическую) функцию для преобразования линейной комбинации входных признаков в вероятность принадлежности объекта к одному из классов (обычно от 0 до 1). Если вероятность превышает определенный порог (например, 0.5), объект относится к положительному классу, иначе — к отрицательному.

Различия между линейной регрессией и логистической регрессией:

Характеристика	Линейная регрессия	Логистическая регрессия
Тип задачи	Регрессия (предсказание непрерывного значения)	Классификация (предсказание вероятности принадлежности к классу)
Выход	Непрерывное число	Вероятность (от 0 до 1)
Функция активации	Нет (линейная функция)	Сигмоидная функция
Функция потерь	MSE (среднеквадратичная ошибка)	Log Loss (кросс-энтропия)

Вероятностная модель: Наивный байесовский классификатор (Naive Bayes Classifier): Семейство алгоритмов классификации, основанных на теореме Байеса с «наивным» предположением о независимости признаков. Это означает, что наличие одного признака не влияет на наличие другого, при условии, что известен класс. Несмотря на это упрощение, наивные байесовские классификаторы часто показывают хорошие результаты, особенно на текстовых данных (например, для фильтрации спама).

11. Метрические модели. Метрики расстояний.

Алгоритм k-ближайших соседей (k-Nearest Neighbor

– kNN).

Метрические модели: Алгоритмы машинного обучения, которые принимают решения на основе расстояний или сходства между точками данных. Они предполагают, что похожие объекты находятся близко друг к другу в пространстве признаков.

Метрики расстояний: Используются для измерения сходства или различия между двумя точками данных:

- **Евклидово расстояние (Euclidean Distance):** Наиболее распространенная метрика, представляющая собой длину отрезка, соединяющего две точки в многомерном пространстве. $d(x, y) = \sqrt{\sum (x_i - y_i)^2}$.
- **Манхэттенское расстояние (Manhattan Distance) / Расстояние городских кварталов:** Сумма абсолютных разностей координат. $d(x, y) = \sum |x_i - y_i|$.
- **Расстояние Чебышева (Chebyshev Distance):** Максимальная абсолютная разность между координатами. $d(x, y) = \max(|x_i - y_i|)$.
- **Расстояние Минковского (Minkowski Distance):** Обобщение Евклидова и Манхэттенского расстояний.

Алгоритм k-ближайших соседей (k-Nearest Neighbor – kNN): Простой, но эффективный алгоритм классификации и регрессии, основанный на метрических моделях. Он относится к «ленивым» алгоритмам, так как не строит явную модель во время обучения, а просто запоминает все обучающие данные.

Как работает kNN:

1. Для нового объекта, который нужно классифицировать (или для которого нужно предсказать значение), алгоритм вычисляет расстояние до всех объектов в обучающей выборке.
2. Выбираются k ближайших соседей к новому объекту.
3. **Для классификации:** Новый объект относится к классу, который является наиболее частым среди его k ближайших соседей (голосование по большинству).
4. **Для регрессии:** Предсказываемое значение для нового объекта является средним (или медианой) значений целевой переменной его k ближайших

соседей.

Выбор k : Является важным гиперпараметром. Малое k делает модель чувствительной к шуму, большое k может сглаживать границы классов и игнорировать локальные особенности.

12. Метод опорных векторов SVM. Линейный SVM. Нелинейный SVM. Ядерный трюк (kernel trick), спрямляющие пространства. SVM с мягким зазором.

Метод опорных векторов (Support Vector Machine, SVM): Мощный алгоритм для задач классификации и регрессии. Основная идея SVM — найти оптимальную разделяющую гиперплоскость, которая максимально отдалена от ближайших точек данных разных классов (эти точки называются опорными векторами).

Линейный SVM: В случае линейно разделимых данных SVM находит гиперплоскость, которая разделяет классы с максимальным зазором (margin). Этот зазор — расстояние от гиперплоскости до ближайших опорных векторов.

Нелинейный SVM: Когда данные не являются линейно разделимыми в исходном пространстве, SVM использует **ядерный трюк (kernel trick)**. Идея заключается в том, чтобы неявно отобразить данные в пространство более высокой размерности, где они становятся линейно разделимыми. При этом нет необходимости явно вычислять координаты точек в этом новом пространстве, достаточно вычислить скалярное произведение с помощью ядерной функции.

Ядерный трюк (Kernel Trick), спрямляющие пространства: Ядерная функция (например, полиномиальное ядро, радиально-базисная функция (RBF) ядро) позволяет вычислять скалярное произведение векторов в пространстве высокой размерности без явного преобразования самих векторов. Это значительно снижает вычислительную сложность и позволяет SVM работать с нелинейными границами решений.

SVM с мягким зазором (Soft Margin SVM): В реальных данных часто встречаются выбросы или перекрывающиеся классы, что делает идеальное линейное разделение невозможным. SVM с мягким зазором позволяет некоторым точкам

данных находиться внутри зазора или даже на неправильной стороне гиперплоскости, вводя штраф за такие нарушения. Это делает модель более устойчивой к шуму и выбросам.

13. Древовидные модели. Алгоритмы, основанные на применении решающих деревьев. Построение дерева решений. Алгоритм CART. Информационный прирост, энтропия, критерий Джини. Эффективность алгоритма дерева решений.

Древовидные модели: Алгоритмы машинного обучения, которые используют структуру дерева для принятия решений. Каждая внутренняя вершина дерева представляет собой проверку значения признака, каждая ветвь — результат проверки, а каждый листовой узел — окончательное решение (класс или значение).

Алгоритмы, основанные на применении решающих деревьев:

- **Деревья решений (Decision Trees):** Базовый алгоритм, который строит дерево для классификации или регрессии.
- **Ансамблевые методы:** Случайный лес (Random Forest), градиентный бустинг (Gradient Boosting), AdaBoost, XGBoost, LightGBM, CatBoost — все они используют решающие деревья в качестве базовых моделей.

Построение дерева решений: Процесс рекурсивного деления данных на подмножества на основе значений признаков. На каждом шаге выбирается признак и пороговое значение, которые наилучшим образом разделяют данные, максимизируя однородность подмножеств.

Алгоритм CART (Classification and Regression Trees): Один из наиболее популярных алгоритмов для построения деревьев решений. Он может использоваться как для классификации, так и для регрессии. CART строит бинарные деревья, где каждая внутренняя вершина имеет ровно две ветви.

Критерии для выбора лучшего разделения (для классификации):

- **Энтропия (Entropy):** Мера неопределенности или хаотичности в наборе данных. Чем ниже энтропия, тем более однородны данные по классам.
- **Информационный прирост (Information Gain):** Разница между энтропией до разделения и взвешенной суммой энтропий после разделения. Алгоритм выбирает разделение, которое максимизирует информационный прирост.
- **Критерий Джини (Gini Impurity):** Мера вероятности неправильной классификации случайно выбранного элемента, если он был случайно помечен в соответствии с распределением классов в подмножестве. Чем ниже критерий Джини, тем чище подмножество.

Эффективность алгоритма дерева решений:

- **Преимущества:** Легко интерпретируются, не требуют масштабирования данных, могут работать с категориальными и числовыми признаками, способны улавливать нелинейные зависимости.
- **Недостатки:** Склонны к переобучению (особенно глубокие деревья), нестабильны (небольшие изменения в данных могут привести к значительному изменению структуры дерева), могут создавать смещенные деревья при несбалансированных данных.

14. Ансамблевое обучение. Soft и Hard Voting (голосование). Ансамбли решающих деревьев: бэггинг, случайный лес (Random Forest). Стекинг и его применение в ансамблевом обучении.

Ансамблевое обучение (Ensemble Learning): Подход в машинном обучении, который объединяет предсказания нескольких моделей (называемых базовыми моделями или слабыми учениками) для получения более точного и устойчивого итогового предсказания, чем у любой отдельной модели. Основная идея — «мудрость толпы».

Soft и Hard Voting (голосование): Методы объединения предсказаний нескольких классификаторов:

- **Hard Voting (Жесткое голосование):** Каждый классификатор голосует за свой класс, и класс, набравший большинство голосов, выбирается как

окончательное предсказание.

- **Soft Voting (Мягкое голосование):** Каждый классификатор предсказывает вероятность принадлежности к каждому классу, и итоговая вероятность для каждого класса суммируется (или усредняется). Класс с наибольшей суммарной вероятностью выбирается как окончательное предсказание. Обычно Soft Voting дает лучшие результаты.

Ансамбли решающих деревьев:

- **Бэггинг (Bagging - Bootstrap Aggregating):** Метод, который строит несколько базовых моделей (обычно деревьев решений) на различных подвыборках обучающих данных, полученных с помощью бутстрапа (выборка с возвращением). Затем предсказания этих моделей усредняются (для регрессии) или голосуются (для классификации). Бэггинг помогает уменьшить дисперсию (variance) модели и снизить переобучение.
- **Случайный лес (Random Forest):** Расширение бэггинга, которое использует не только случайные подвыборки данных, но и случайные подвыборки признаков при построении каждого дерева. Это делает деревья более независимыми друг от друга и дополнительно снижает переобучение. Случайный лес очень популярен благодаря своей высокой точности и устойчивости.

Стекинг (Stacking): Более сложный ансамблевый метод, который использует мета-модель (или «ученика второго уровня») для объединения предсказаний нескольких базовых моделей (учеников первого уровня). Базовые модели обучаются на обучающей выборке, а их предсказания используются как входные признаки для мета-модели, которая затем обучается предсказывать целевую переменную. Стекинг часто дает очень высокие результаты, но более сложен в реализации.

15. Ансамбли моделей: бустинг, AdaBoost, градиентный бустинг. Библиотеки градиентного

бустинга над решающими деревьями (CatBoost, LightGBM, XGBoost, SketchBoost).

Бустинг (Boosting): Семейство ансамблевых методов, которые строят базовые модели последовательно, причем каждая последующая модель пытается исправить ошибки предыдущих. Основная идея — фокусироваться на тех данных, где предыдущие модели ошиблись. Бустинг помогает уменьшить смещение (bias) модели.

AdaBoost (Adaptive Boosting): Один из первых и наиболее известных алгоритмов бустинга. Он последовательно обучает слабые классификаторы (обычно неглубокие деревья решений), присваивая больший вес неправильно классифицированным примерам на каждой итерации. Окончательное предсказание формируется взвешенным голосованием всех слабых классификаторов.

Градиентный бустинг (Gradient Boosting): Более общий и мощный подход к бустингу. Вместо того чтобы корректировать веса примеров, как в AdaBoost, градиентный бустинг строит каждую новую базовую модель для предсказания «остатков» (residuals) или градиентов функции потерь предыдущих моделей. Это позволяет алгоритму минимизировать любую дифференцируемую функцию потерь. Чаще всего в качестве базовых моделей используются деревья решений.

Библиотеки градиентного бустинга над решающими деревьями: Это высокопроизводительные реализации алгоритмов градиентного бустинга, оптимизированные для скорости и точности, которые стали стандартом де-факто во многих задачах машинного обучения:

- **XGBoost (eXtreme Gradient Boosting):** Очень популярная и эффективная библиотека, известная своей скоростью, масштабируемостью и точностью. Поддерживает параллельные вычисления и различные типы регуляризации.
- **LightGBM (Light Gradient Boosting Machine):** Разработан Microsoft, отличается еще большей скоростью обучения и меньшим потреблением памяти по сравнению с XGBoost, особенно на больших наборах данных. Использует алгоритм GOSS (Gradient-based One-Side Sampling) и EFB (Exclusive Feature Bundling).

- **CatBoost (Categorical Boosting):** Разработан Яндекс, особенно хорошо работает с категориальными признаками благодаря встроенным механизмам их обработки. Также известен своей устойчивостью к переобучению и хорошими результатами «из коробки».
- **SketchBoost:** Менее известная, но также эффективная библиотека, использующая методы скетчинга для ускорения обучения на больших данных.

16. Обучение без учителя. Цели обучения без учителя. Задача кластеризации, постановка задачи кластеризации. Методы оценки кластеризации.

Обучение без учителя (Unsupervised Learning): Тип машинного обучения, где алгоритмы работают с неразмеченными данными, то есть без заранее известных правильных ответов. Цель состоит в том, чтобы найти скрытые структуры, закономерности или представления в данных.

Цели обучения без учителя:

- **Кластеризация:** Группировка похожих объектов.
- **Понижение размерности:** Уменьшение количества признаков.
- **Обнаружение аномалий:** Выявление необычных или редких объектов.
- **Ассоциативные правила:** Поиск взаимосвязей между элементами (например, «если купил X, то купит и Y»).

Задача кластеризации: Процесс разделения набора объектов на группы (кластеры) таким образом, чтобы объекты в одном кластере были более похожи друг на друга, чем на объекты в других кластерах. В отличие от классификации, кластеры не predetermined, и алгоритм сам их находит.

Постановка задачи кластеризации: Дано множество объектов без меток классов. Требуется разбить это множество на k кластеров, где k может быть задано заранее или определено алгоритмом. Цель — максимизировать внутрикластерное сходство и минимизировать межкластерное сходство.

Методы оценки кластеризации: Поскольку нет «правильных» меток, оценка кластеризации сложнее, чем классификации. Методы делятся на:

- **Внутренние метрики (Internal Evaluation):** Оценивают качество кластеризации на основе только самих данных и полученных кластеров, без использования внешних меток.
 - **Индекс силуэта (Silhouette Score):** Измеряет, насколько объект похож на свой собственный кластер по сравнению с другими кластерами. Значения от -1 до 1. Чем ближе к 1, тем лучше кластеризация.
 - **Индекс Дэвиса-Болдина (Davies-Bouldin Index):** Отношение внутрикластерного расстояния к межкластерному. Чем меньше, тем лучше.
 - **Индекс Калински-Харабаса (Calinski-Harabasz Index):** Отношение дисперсии между кластерами к дисперсии внутри кластеров. Чем больше, тем лучше.
- **Внешние метрики (External Evaluation):** Используются, когда доступны истинные метки классов (хотя в обучении без учителя их обычно нет, но они могут быть использованы для сравнения).
 - **Adjusted Rand Index (ARI):** Мера сходства между двумя разбиениями на кластеры, скорректированная с учетом случайности.
 - **Normalized Mutual Information (NMI):** Мера взаимной информации между истинными метками и кластерами, нормализованная.

17. Задача кластеризации. Алгоритмы кластеризации. Алгоритм k-средних. Иерархическая кластеризация. Affinity Propagation.

Алгоритмы кластеризации:

Алгоритм k-средних (k-Means): Один из самых популярных и простых алгоритмов кластеризации. Он относится к центроидным алгоритмам.

Как работает k-Means:

1. **Инициализация:** Случайным образом выбираются k центроидов (центров кластеров).

2. **Присвоение:** Каждый объект данных присваивается к ближайшему центроиду.
3. **Обновление:** Центроиды пересчитываются как среднее арифметическое всех объектов, принадлежащих их кластеру.
4. **Повторение:** Шаги 2 и 3 повторяются до тех пор, пока центроиды не перестанут значительно меняться или не будет достигнуто максимальное количество итераций.

Преимущества k-Means: Прост в реализации, быстр на больших наборах данных. **Недостатки k-Means:** Требуется заранее задавать количество кластеров `k`, чувствителен к начальному выбору центроидов и к выбросам, плохо работает с кластерами неправильной формы.

Иерархическая кластеризация (Hierarchical Clustering): Строит иерархию кластеров, которая может быть представлена в виде дендрограммы. Существует два основных типа:

- **Агломеративная (Agglomerative):** «Снизу вверх». Каждый объект сначала считается отдельным кластером, затем ближайшие кластеры последовательно объединяются, пока не останется один большой кластер.
- **Дивизивная (Divisive):** «Сверху вниз». Все объекты начинаются в одном большом кластере, который затем рекурсивно делится на более мелкие кластеры.

Преимущества иерархической кластеризации: Не требуется заранее задавать `k`, позволяет визуализировать структуру кластеров с помощью дендрограммы. **Недостатки:** Вычислительно затратно для больших данных, сложно определить оптимальное количество кластеров по дендрограмме.

Affinity Propagation: Алгоритм кластеризации, который не требует заранее задавать количество кластеров. Он находит «образцы» (exemplars) — точки данных, которые являются представителями кластеров. Алгоритм работает путем обмена «сообщениями» между точками данных до тех пор, пока не будут определены образцы и соответствующие им кластеры.

Преимущества Affinity Propagation: Не нужно задавать `k`, может находить кластеры произвольной формы. **Недостатки:** Вычислительно затратен для больших данных, чувствителен к параметрам `preference` и `damping`.

18. Алгоритм DBSCAN, пространственные данные. Гауссовы смеси, ЕМ-алгоритм. Преимущества и недостатки методов кластеризации.

Алгоритм DBSCAN (Density-Based Spatial Clustering of Applications with Noise):

Алгоритм кластеризации, основанный на плотности. Он группирует плотно расположенные точки данных, помечая точки, которые лежат в областях с низкой плотностью, как выбросы. DBSCAN не требует заранее задавать количество кластеров и может находить кластеры произвольной формы.

Как работает DBSCAN:

1. Для каждой точки данных алгоритм определяет ее ϵ -окрестность (радиус ϵ).
2. Если в ϵ -окрестности точки содержится не менее minPts других точек, то эта точка считается **ядровой (core point)**.
3. Все ядровые точки и точки, находящиеся в их ϵ -окрестности, формируют кластер.
4. Точки, которые не являются ядровыми, но находятся в ϵ -окрестности ядровой точки, называются **граничными (border points)**.
5. Точки, которые не являются ни ядровыми, ни граничными, считаются **шумом (noise points)** или выбросами.

Преимущества DBSCAN: Не требует k , находит кластеры произвольной формы, устойчив к шуму. **Недостатки:** Плохо работает с кластерами разной плотности, чувствителен к выбору параметров ϵ и minPts .

Пространственные данные: Данные, которые имеют географическую или пространственную привязку (например, координаты GPS, местоположение магазинов). DBSCAN хорошо подходит для кластеризации таких данных.

Гауссовы смеси (Gaussian Mixture Models, GMM): Вероятностная модель, которая предполагает, что данные генерируются из смеси нескольких гауссовых распределений (нормальных распределений). GMM пытается найти параметры этих гауссовых распределений (среднее, дисперсия, веса), чтобы наилучшим образом объяснить наблюдаемые данные. GMM может использоваться для

кластеризации, где каждый кластер соответствует одному гауссову распределению.

ЕМ-алгоритм (Expectation-Maximization Algorithm): Итеративный алгоритм, используемый для нахождения оценок максимального правдоподобия параметров вероятностных моделей, когда модель зависит от скрытых (ненаблюдаемых) переменных. ЕМ-алгоритм состоит из двух шагов:

1. **Е-шаг (Expectation):** Вычисление ожидаемых значений скрытых переменных на основе текущих параметров модели.
2. **М-шаг (Maximization):** Обновление параметров модели для максимизации правдоподобия, используя ожидаемые значения скрытых переменных.

ЕМ-алгоритм часто используется для обучения GMM.

Преимущества и недостатки методов кластеризации:

Метод	Преимущества	Недостатки
k-Means	Прост, быстр, эффективен для сферических кластеров.	Требуется k , чувствителен к выбросам, плохо для несферических кластеров.
Иерархическая	Не требует k , визуализация дендрограммой.	Вычислительно затратно, сложно определить k .
DBSCAN	Не требует k , находит кластеры произвольной формы, устойчив к шуму.	Плохо для кластеров разной плотности, чувствителен к параметрам.
GMM	Гибкий, может находить эллиптические кластеры, дает вероятности принадлежности.	Вычислительно затратен, чувствителен к инициализации, требует k .

19. Глубокое обучение. Нейронные сети (НС).

Модель искусственного нейрона. Функции активации. Понятие батча и эпохи. Полносвязные нейронные сети (FCNN). Функции ошибки и

обучение с помощью обратного распространения градиента.

Глубокое обучение (Deep Learning): Подраздел машинного обучения, основанный на искусственных нейронных сетях с множеством слоев (глубокие сети). Глубокое обучение позволяет моделям автоматически извлекать сложные признаки из сырых данных, что делает его особенно эффективным для задач компьютерного зрения, обработки естественного языка и распознавания речи.

Нейронные сети (НС): Вычислительные модели, вдохновленные структурой и функционированием биологического мозга. Они состоят из взаимосвязанных узлов (нейронов), организованных в слои.

Модель искусственного нейрона (Perceptron): Базовая единица нейронной сети. Он принимает несколько входных сигналов, умножает их на соответствующие веса, суммирует результаты, добавляет смещение (bias) и пропускает полученную сумму через **функцию активации**, чтобы произвести выходной сигнал.

Функции активации (Activation Functions): Нелинейные функции, применяемые к взвешенной сумме входов нейрона. Они вводят нелинейность в модель, позволяя нейронным сетям учиться сложным, нелинейным зависимостям. Примеры:

- **Сигмоида (Sigmoid):** $\sigma(x) = 1 / (1 + e^{(-x)})$. Выход от 0 до 1, используется в выходных слоях для бинарной классификации.
- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. Самая популярная функция активации, проста в вычислениях и помогает избежать проблемы затухающих градиентов.
- **Tanh (Hyperbolic Tangent):** $f(x) = (e^x - e^{(-x)}) / (e^x + e^{(-x)})$. Выход от -1 до 1.
- **Softmax:** Используется в выходных слоях для многоклассовой классификации, преобразует выходы в вероятности, сумма которых равна 1.

Понятие батча (Batch) и эпохи (Epoch):

- **Бэтч (Batch):** Подмножество обучающих данных, которое используется для одной итерации обновления весов модели. Использование батчей вместо всей выборки ускоряет обучение и помогает избежать локальных минимумов.
- **Эпоха (Epoch):** Один полный проход по всей обучающей выборке. В течение одной эпохи модель видит каждый обучающий пример один раз.

Полносвязные нейронные сети (Fully Connected Neural Networks, FCNN) / Многослойный перцептрон (Multilayer Perceptron, MLP): Тип нейронной сети, где каждый нейрон одного слоя соединен со всеми нейронами следующего слоя. Состоят из входного слоя, одного или нескольких скрытых слоев и выходного слоя.

Функции ошибки (Loss Functions) / Функции стоимости (Cost Functions): Измеряют разницу между предсказанными и фактическими значениями. Цель обучения — минимизировать эту функцию.

- **MSE (Mean Squared Error):** Для регрессии.
- **Binary Cross-Entropy:** Для бинарной классификации.
- **Categorical Cross-Entropy:** Для многоклассовой классификации.

Обучение с помощью обратного распространения градиента (Backpropagation): Основной алгоритм обучения нейронных сетей. Он состоит из двух фаз:

1. **Прямой проход (Forward Pass):** Входные данные проходят через сеть, и вычисляется выходное значение и значение функции ошибки.
2. **Обратный проход (Backward Pass):** Градиенты функции ошибки по отношению к весам сети вычисляются, начиная с выходного слоя и двигаясь назад к входному. Эти градиенты затем используются для обновления весов сети с помощью алгоритма оптимизации (например, градиентного спуска).

20. Базовая архитектура нейронных сетей.

Перцептрон Розенблатта. Многослойный персептрон (MLP). Реализация многослойного

перцептрона с использованием библиотеки Scikit-learn.

Базовая архитектура нейронных сетей: Нейронные сети обычно состоят из слоев нейронов:

- **Входной слой (Input Layer):** Принимает входные данные. Количество нейронов равно количеству признаков.
- **Скрытые слои (Hidden Layers):** Один или несколько слоев между входным и выходным. Здесь происходит основная обработка и извлечение признаков. Количество нейронов и слоев может варьироваться.
- **Выходной слой (Output Layer):** Производит окончательное предсказание. Количество нейронов зависит от типа задачи (1 для бинарной классификации/регрессии, несколько для многоклассовой классификации).

Перцептрон Розенблатта (Rosenblatt's Perceptron): Самая простая форма нейронной сети, разработанная Фрэнком Розенблаттом в 1957 году. Это однослойная нейронная сеть, способная решать задачи линейной классификации. Он принимает бинарные входы, умножает их на веса, суммирует и пропускает через пороговую функцию активации для получения бинарного выхода. Перцептрон может обучаться с помощью алгоритма перцептрона.

Многослойный перцептрон (Multilayer Perceptron, MLP): Нейронная сеть, состоящая из одного входного слоя, одного или нескольких скрытых слоев и одного выходного слоя. В отличие от однослойного перцептрона, MLP с нелинейными функциями активации способен решать нелинейные задачи классификации и регрессии. Каждый нейрон в одном слое полностью соединен со всеми нейронами в следующем слое.

Реализация многослойного перцептрона с использованием библиотеки Scikit-learn: Scikit-learn предоставляет класс `MLPClassifier` для задач классификации и `MLPRegressor` для задач регрессии. Пример использования `MLPClassifier`:

```

from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Загрузка данных
iris = load_iris()
X, y = iris.data, iris.target

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Масштабирование данных (важно для НС)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Инициализация и обучение MLPClassifier
# hidden_layer_sizes=(100,) означает один скрытый слой из 100 нейронов
# max_iter=500 - максимальное количество итераций (эпох)
mlp = MLPClassifier(hidden_layer_sizes=(100,), max_iter=500,
random_state=42)
mlp.fit(X_train_scaled, y_train)

# Оценка модели
accuracy = mlp.score(X_test_scaled, y_test)
print(f"Точность MLPClassifier: {accuracy:.2f}")

```

21. Полносвязные нейронные сети (FCNN).

Фреймворк TensorFlow и API Keras для построения FCNN. Решение задачи регрессии и классификации с помощью FCNN.

Полносвязные нейронные сети (Fully Connected Neural Networks, FCNN): Это синоним многослойного перцептрона (MLP). Как уже упоминалось, это сети, где каждый нейрон одного слоя соединен со всеми нейронами следующего слоя. Они являются базовым строительным блоком для многих более сложных архитектур нейронных сетей.

Фреймворк TensorFlow и API Keras:

- **TensorFlow:** Мощная библиотека с открытым исходным кодом, разработанная Google, для численных вычислений и крупномасштабного машинного обучения. Она предоставляет низкоуровневые API для построения и обучения нейронных сетей.
- **Keras:** Высокоуровневый API для построения и обучения моделей глубокого обучения. Keras был интегрирован в TensorFlow (как `tf.keras`) и стал де-факто стандартом для быстрого прототипирования и разработки нейронных сетей благодаря своей простоте и модульности. Keras позволяет легко определять слои, функции активации, оптимизаторы и функции потерь.

Построение FCNN с использованием TensorFlow/Keras:

```

import tensorflow as tf
from tensorflow import keras
from sklearn.datasets import make_classification, make_regression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# --- Решение задачи классификации с помощью FCNN ---
# Генерация синтетических данных для классификации
X_clf, y_clf = make_classification(n_samples=1000, n_features=20,
n_classes=2, random_state=42)
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(X_clf,
y_clf, test_size=0.2, random_state=42)

# Масштабирование данных
scaler_clf = StandardScaler()
X_train_clf_scaled = scaler_clf.fit_transform(X_train_clf)
X_test_clf_scaled = scaler_clf.transform(X_test_clf)

# Построение FCNN для классификации
model_clf = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=
(X_train_clf_scaled.shape[1],)), # Входной и первый скрытый слой
    keras.layers.Dropout(0.3), # Слой для предотвращения переобучения
    keras.layers.Dense(64, activation='relu'), # Второй скрытый слой
    keras.layers.Dense(1, activation='sigmoid') # Выходной слой для
бинарной классификации
])

# Компиляция модели
model_clf.compile(optimizer='adam', loss='binary_crossentropy', metrics=
['accuracy'])

# Обучение модели
model_clf.fit(X_train_clf_scaled, y_train_clf, epochs=10, batch_size=32,
verbose=0)

# Оценка модели
loss_clf, accuracy_clf = model_clf.evaluate(X_test_clf_scaled, y_test_clf,
verbose=0)
print(f"\nFCNN для классификации - Точность: {accuracy_clf:.2f}")

# --- Решение задачи регрессии с помощью FCNN ---
# Генерация синтетических данных для регрессии
X_reg, y_reg = make_regression(n_samples=1000, n_features=20,
random_state=42)

```



```

X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg,
y_reg, test_size=0.2, random_state=42)

# Масштабирование данных
scaler_reg = StandardScaler()
X_train_reg_scaled = scaler_reg.fit_transform(X_train_reg)
X_test_reg_scaled = scaler_reg.transform(X_test_reg)

# Построение FCNN для регрессии
model_reg = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=
(X_train_reg_scaled.shape[1],)),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(1) # Выходной слой для регрессии (без функции
активации или с линейной)
])

# Компиляция модели
model_reg.compile(optimizer='adam', loss='mse', metrics=['mae'])

# Обучение модели
model_reg.fit(X_train_reg_scaled, y_train_reg, epochs=10, batch_size=32,
verbose=0)

# Оценка модели
loss_reg, mae_reg = model_reg.evaluate(X_test_reg_scaled, y_test_reg,
verbose=0)
print(f"FCNN для регрессии - MAE: {mae_reg:.2f}")

```

22. Работа с изображениями с помощью сверточных НС. Общая структура сверточного слоя. Операции сверток, max-pooling. Параметры сверточного слоя.

Работа с изображениями с помощью сверточных НС (Convolutional Neural Networks, CNN): CNN — это специализированный тип нейронных сетей, который особенно эффективен для обработки данных с сеточной топологией, таких как изображения. Они автоматически извлекают иерархические признаки из

изображений, начиная от простых (края, углы) до более сложных (части объектов, целые объекты).

Общая структура сверточного слоя (Convolutional Layer): Сверточный слой является основным строительным блоком CNN. Он состоит из набора обучаемых фильтров (ядер), которые «свертываются» с входным изображением (или картой признаков) для создания карт признаков (feature maps).

Операции сверток (Convolution Operations):

1. **Фильтр (Kernel/Filter):** Небольшая матрица весов, которая скользит по входному изображению.
2. **Свертка:** На каждой позиции фильтр умножается поэлементно на соответствующую область входного изображения, и результаты суммируются в одно значение. Это значение записывается в выходную карту признаков.
3. **Карта признаков (Feature Map):** Выход сверточного слоя, который показывает, где в изображении были обнаружены определенные признаки, на которые настроен фильтр.

Max-pooling (Макс-пулинг): Операция, которая обычно следует за сверточным слоем. Ее цель — уменьшить пространственный размер карты признаков, тем самым сокращая количество параметров и вычислений в сети, а также помогая сделать модель более устойчивой к небольшим сдвигам во входном изображении. Max-pooling выбирает максимальное значение из каждого подрегиона (окна) карты признаков.

Параметры сверточного слоя:

- **Количество фильтров (Number of Filters):** Определяет количество карт признаков, которые будут сгенерированы слоем. Каждый фильтр учится распознавать свой уникальный признак.
- **Размер фильтра (Filter Size/Kernel Size):** Размеры матрицы фильтра (например, 3x3, 5x5).
- **Шаг (Stride):** Количество пикселей, на которое фильтр смещается при каждом шаге по входному изображению. Большой шаг приводит к уменьшению размера выходной карты признаков.

- **Отступы (Padding):** Добавление нулей по краям входного изображения, чтобы контролировать пространственный размер выходной карты признаков и предотвратить потерю информации по краям. `Same padding` сохраняет размер, `valid padding` уменьшает его.
- **Функция активации (Activation Function):** Применяется к выходу свертки (обычно ReLU).

23. Архитектура сверточных НС. Популярные архитектуры сверточных НС: AlexNet, VGG, Inception (GoogLeNet), ResNet. Трансферное обучение.

Архитектура сверточных НС (CNN Architecture): Типичная CNN состоит из чередующихся сверточных слоев, слоев активации (ReLU), слоев пулинга (Max-pooling), за которыми следуют один или несколько полносвязных слоев и, наконец, выходной слой. Глубокие CNN имеют множество таких блоков, что позволяет им извлекать все более абстрактные и сложные признаки.

Популярные архитектуры сверточных НС:

- **AlexNet (2012):** Одна из первых глубоких CNN, которая показала прорывные результаты в конкурсе ImageNet. Состояла из 5 сверточных слоев, 3 слоев пулинга и 3 полносвязных слоев. Ввела использование ReLU и Dropout.
- **VGG (Visual Geometry Group) (2014):** Отличается простотой и однородностью архитектуры, используя только сверточные слои 3x3 и Max-pooling 2x2. Различные версии VGG (например, VGG16, VGG19) различаются глубиной сети. Показала, что глубина сети важна для производительности.
- **Inception (GoogLeNet) (2014):** Ввела «Inception-модули», которые позволяют сети выполнять свертки с разными размерами фильтров и пулинг параллельно на одном уровне, а затем объединять их выходы. Это позволяет сети эффективно захватывать признаки на разных масштабах и значительно сокращает количество параметров.
- **ResNet (Residual Network) (2015):** Ввела концепцию «остаточных связей» (skip connections), которые позволяют градиентам течь напрямую через несколько слоев, решая проблему затухающих градиентов в очень глубоких

сетях. Это позволило строить сети с сотнями слоев и достигать выдающихся результатов.

Трансферное обучение (Transfer Learning): Методика, при которой модель, обученная на одной задаче (например, на большом наборе данных ImageNet для классификации изображений), используется в качестве отправной точки для новой, связанной задачи. Вместо того чтобы обучать новую модель с нуля, можно взять предобученную модель, «заморозить» ее ранние слои (которые извлекают общие признаки) и переобучить только последние слои (которые специализируются на конкретной задаче). Это очень эффективно, когда доступно мало данных для новой задачи, и значительно сокращает время и вычислительные ресурсы для обучения.

24. Задача понижения размерности. Метод главных компонент (PCA). Алгоритм PCA. Ядровый PCA.

Задача понижения размерности (Dimensionality Reduction): Процесс уменьшения количества случайных переменных (признаков) в наборе данных путем получения набора главных переменных. Цель — сохранить как можно больше важной информации, удалив избыточные или менее значимые признаки. Это помогает бороться с «проклятием размерности», ускоряет обучение моделей и улучшает их обобщающую способность, а также облегчает визуализацию данных.

Метод главных компонент (Principal Component Analysis, PCA): Один из наиболее популярных линейных методов понижения размерности. PCA преобразует исходные признаки в новый набор некоррелированных признаков, называемых главными компонентами. Эти компоненты упорядочены по убыванию дисперсии, которую они объясняют в данных, то есть первая главная компонента объясняет наибольшую дисперсию, вторая — следующую по величине и так далее.

Алгоритм PCA:

1. **Стандартизация данных:** Масштабирование признаков так, чтобы они имели нулевое среднее и единичную дисперсию (если признаки имеют разные масштабы).

2. **Вычисление ковариационной матрицы:** Матрица, показывающая ковариацию между всеми парами признаков.
3. **Вычисление собственных векторов и собственных значений:** Собственные векторы ковариационной матрицы представляют направления главных компонент, а собственные значения — величину дисперсии вдоль этих направлений.
4. **Сортировка:** Собственные векторы сортируются в порядке убывания соответствующих им собственных значений.
5. **Выбор главных компонент:** Выбирается k собственных векторов с наибольшими собственными значениями, где k — желаемое количество измерений.
6. **Проекция данных:** Исходные данные проецируются на новое подпространство, образованное выбранными главными компонентами.

Ядровый PCA (Kernel PCA): Расширение стандартного PCA, которое позволяет выполнять нелинейное понижение размерности. Используя ядерный трюк (аналогично SVM), Kernel PCA неявно отображает данные в пространство более высокой размерности, где затем применяется линейный PCA. Это позволяет находить нелинейные главные компоненты и эффективно работать с данными, которые не являются линейно разделимыми в исходном пространстве.

25. Нелинейные методы снижения размерности: t-SNE, Isomap, UMAP. Методы выбора признаков (Feature Selection).

Нелинейные методы снижения размерности: В отличие от PCA, который ищет линейные преобразования, эти методы способны находить и сохранять нелинейные структуры в данных, что часто более актуально для реальных сложных наборов данных.

- **t-SNE (t-Distributed Stochastic Neighbor Embedding):** Популярный нелинейный метод снижения размерности, особенно хорошо подходящий для визуализации высокоразмерных данных в 2D или 3D. t-SNE пытается сохранить локальные структуры данных, то есть объекты, которые были близки в высокоразмерном пространстве, остаются близкими и в

низкоразмерном. Он часто используется для визуализации кластеров в данных.

- **Isomap (Isometric Mapping):** Алгоритм, который пытается сохранить геодезические расстояния (расстояния вдоль многообразия) между точками данных. Он строит граф ближайших соседей, а затем использует алгоритм кратчайшего пути для оценки геодезических расстояний, после чего применяет многомерное шкалирование (MDS) для получения низкоразмерного представления.
- **UMAP (Uniform Manifold Approximation and Projection):** Относительно новый и очень эффективный нелинейный метод снижения размерности. UMAP часто быстрее t-SNE и лучше сохраняет как локальные, так и глобальные структуры данных. Он основан на теории римановых многообразий и топологическом анализе данных.

Методы выбора признаков (Feature Selection): Процесс выбора подмножества релевантных признаков из исходного набора данных. Цель — уменьшить количество признаков, сохранив или улучшив производительность модели, уменьшить вычислительные затраты и улучшить интерпретируемость модели. В отличие от понижения размерности, где создаются новые признаки, выбор признаков оставляет только подмножество исходных признаков.

Типы методов выбора признаков:

- **Методы фильтрации (Filter Methods):** Оценивают релевантность признаков независимо от модели машинного обучения, используя статистические меры (например, корреляция, хи-квадрат, информационный прирост). Признаки ранжируются, и выбираются лучшие.
 - **Преимущества:** Быстрые, простые, не зависят от модели.
 - **Недостатки:** Не учитывают взаимодействия между признаками и влияние на конкретную модель.
- **Методы обертывания (Wrapper Methods):** Используют модель машинного обучения для оценки подмножеств признаков. Они итеративно выбирают подмножества признаков, обучают модель и оценивают ее производительность. Примеры: пошаговый отбор (forward/backward selection), рекурсивное исключение признаков (Recursive Feature Elimination, RFE).

- **Преимущества:** Учитывают взаимодействие признаков и специфику модели.
- **Недостатки:** Вычислительно затратны, склонны к переобучению на тестовой выборке.
- **Встроенные методы (Embedded Methods):** Методы, которые выполняют отбор признаков в процессе обучения модели. Примеры: Lasso-регрессия (L1-регуляризация), деревья решений и ансамбли на их основе (которые могут оценивать важность признаков).
 - **Преимущества:** Эффективны, учитывают взаимодействие признаков, интегрированы в процесс обучения.
 - **Недостатки:** Зависят от конкретной модели.

Вопрос 1: Основные понятия машинного обучения (обучение с учителем, без учителя, с подкреплением). Области применения.

Обучение с учителем (Supervised Learning): Это тип машинного обучения, где модель обучается на размеченных данных, то есть на данных, для которых уже известны правильные ответы (метки). Цель — научить модель предсказывать эти метки для новых, невидимых данных. Примеры задач: **классификация** (предсказание категории, например, спам/не спам) и **регрессия** (предсказание числового значения, например, цены дома).

Обучение без учителя (Unsupervised Learning): В этом подходе модель работает с неразмеченными данными, то есть без заранее известных ответов. Цель — найти скрытые закономерности, структуры или взаимосвязи в данных. Примеры задач: **кластеризация** (группировка похожих объектов, например, сегментация клиентов) и **снижение размерности** (упрощение данных, сохраняя при этом важную информацию).

Обучение с подкреплением (Reinforcement Learning): Это подход, при котором агент учится принимать решения в интерактивной среде, получая «награды» за правильные действия и «штрафы» за неправильные. Цель — максимизировать общую награду за длительный период. Примеры: обучение роботов ходить, игры (например, AlphaGo).

Области применения машинного обучения:

- **Медицина:** диагностика заболеваний, разработка лекарств.

- **Финансы:** обнаружение мошенничества, прогнозирование рынков.
- **Маркетинг:** персонализированные рекомендации, анализ поведения клиентов.
- **Автономные системы:** беспилотные автомобили, робототехника.
- **Обработка естественного языка:** перевод, анализ тональности текста.
- **Компьютерное зрение:** распознавание лиц, объектов.

Вопрос 2: Линейная регрессия: математическая модель, метод наименьших квадратов, аналитическое решение.

Линейная регрессия — это один из самых простых и широко используемых алгоритмов машинного обучения для предсказания непрерывных числовых значений. Она предполагает линейную зависимость между входными признаками (независимыми переменными) и целевой переменной (зависимой переменной).

Математическая модель: Модель линейной регрессии выражается уравнением прямой (или гиперплоскости в многомерном случае):

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n + \epsilon$$

Где:

- y — предсказываемое значение (целевая переменная).
- b_0 — свободный член (пересечение с осью Y).
- $b_1, b_2, \dots, b_n$ — коэффициенты регрессии (веса), показывающие, как сильно каждый признак x_i влияет на y .
- $x_1, x_2, \dots, x_n$ — входные признаки.
- ϵ — случайная ошибка (шум).

В векторной форме это можно записать как:

$$h_{\theta}(x) = \theta^T x$$

Где $h_{\theta}(x)$ — предсказание модели, θ — вектор параметров (коэффициентов), а x — вектор входных признаков (с добавленной единицей для b_0).

Метод наименьших квадратов (Ordinary Least Squares, OLS): Цель линейной регрессии — найти такие коэффициенты $b_0, b_1, \dots, b_n$, которые минимизируют сумму квадратов разностей между фактическими значениями y и предсказанными моделью $h_\theta(x)$. Эта сумма квадратов ошибок называется **функцией потерь** (или функцией стоимости, Cost Function):

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2$$

Где m — количество обучающих примеров.

Метод наименьших квадратов находит оптимальные коэффициенты, которые минимизируют эту функцию потерь.

Аналитическое решение (Normal Equation): Для линейной регрессии существует аналитическое решение, которое позволяет найти оптимальные коэффициенты θ без использования итерационных методов (как, например, градиентный спуск). Это решение называется **нормальным уравнением**:

$$\theta = (X^T X)^{-1} X^T y$$

Где:

- θ — вектор оптимальных коэффициентов.
- X — матрица признаков (каждая строка — один пример, каждый столбец — один признак, с добавленным столбцом единиц для свободного члена).
- y — вектор целевых значений.
- X^T — транспонированная матрица X .
- $(X^T X)^{-1}$ — обратная матрица произведения $X^T X$.

Это решение эффективно для небольших и средних наборов данных, но может быть вычислительно дорогим для очень больших матриц из-за необходимости вычисления обратной матрицы.

Вопрос 3: Градиентный спуск: идея, алгоритм, скорость обучения, стохастический градиентный спуск.

Градиентный спуск (Gradient Descent): Это общий оптимизационный алгоритм, используемый для поиска минимума функции (в нашем случае — функции потерь). Идея заключается в итеративном движении в направлении,

противоположном градиенту функции, поскольку градиент указывает на направление наибольшего роста. Представьте, что вы спускаетесь с горы в тумане: вы делаете шаг в направлении наибольшего уклона вниз.

Алгоритм:

1. **Инициализация:** Выбираются случайные начальные значения для параметров модели (коэффициентов θ).
2. **Итерации:** Повторяются следующие шаги до сходимости (пока параметры не перестанут значительно меняться или не будет достигнуто максимальное количество итераций):
 - Вычисляется градиент функции потерь по каждому параметру. Градиент показывает, насколько сильно изменится функция потерь при небольшом изменении параметра.
 - Параметры обновляются: из текущего значения параметра вычитается произведение градиента на **скорость обучения** (learning rate).

Формула обновления для каждого параметра θ_j :

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Где:

- α — скорость обучения.
- $\frac{\partial}{\partial \theta_j} J(\theta)$ — частная производная функции потерь по параметру θ_j .

Скорость обучения (Learning Rate, α): Это гиперпараметр, который определяет размер шага, который делается при каждой итерации градиентного спуска.

- **Слишком большая скорость обучения:** может привести к тому, что алгоритм будет «перепрыгивать» минимум и расходиться (не сходиться к оптимальному решению).
- **Слишком маленькая скорость обучения:** алгоритм будет сходиться очень медленно, требуя большого количества итераций. Выбор оптимальной скорости обучения критичен для эффективности алгоритма.

Стохастический градиентный спуск (Stochastic Gradient Descent, SGD): В классическом (пакетном) градиентном спуске градиент вычисляется по **всем**

обучающим примерам на каждой итерации. Это может быть очень медленно для больших наборов данных.

SGD решает эту проблему, вычисляя градиент и обновляя параметры, используя **только один случайно выбранный обучающий пример** на каждой итерации. Это делает процесс обновления намного быстрее, особенно на больших данных.

Преимущества SGD:

- Значительно быстрее для больших наборов данных.
- Может помочь избежать локальных минимумов в невыпуклых функциях потерь, так как его «шумное» обновление позволяет ему «выпрыгивать» из них.

Недостатки SGD:

- Путь к минимуму более «шумный» и менее стабильный, чем у пакетного градиентного спуска.
- Может колебаться вокруг минимума, не достигая его точно.

Существуют также варианты, такие как **мини-пакетный градиентный спуск (Mini-batch Gradient Descent)**, который использует небольшую подвыборку данных (мини-пакет) для вычисления градиента, что является компромиссом между пакетным градиентным спуском и SGD.

Вопрос 4: Полиномиальная регрессия. Переобучение и недообучение. Регуляризация (L1, L2).

Полиномиальная регрессия (Polynomial Regression): Это форма регрессионного анализа, в которой связь между независимой переменной x и зависимой переменной y моделируется как n -степенный полином. Она используется, когда линейная модель недостаточно хорошо описывает данные, и есть основания полагать, что зависимость нелинейная.

Математическая модель:

$$y = b_0 + b_1x + b_2x^2 + \dots + b_nx^n + \epsilon$$

Где n — степень полинома. Чем выше степень, тем более сложную зависимость может уловить модель. Полиномиальная регрессия по сути является частным

случае линейной регрессии, где в качестве признаков используются степени исходного признака.

Переобучение (Overfitting) и Недообучение (Underfitting): Это две основные проблемы, с которыми сталкиваются модели машинного обучения.

- **Недообучение (Underfitting):** Происходит, когда модель слишком проста и не может уловить основные закономерности в данных. Она плохо работает как на обучающих, так и на новых данных. Причины: слишком простая модель (например, линейная модель для нелинейных данных), недостаточное количество признаков, слишком сильная регуляризация.
- **Переобучение (Overfitting):** Происходит, когда модель слишком сложна и «запоминает» обучающие данные, включая шум и случайные флуктуации, вместо того чтобы улавливать общие закономерности. Такая модель отлично работает на обучающих данных, но очень плохо на новых, невидимых данных. Причины: слишком сложная модель (например, полином высокой степени), слишком много признаков, недостаточное количество обучающих данных, отсутствие регуляризации.

Регуляризация (Regularization): Это методы, используемые для предотвращения переобучения путем добавления штрафа к функции потерь за большие значения коэффициентов модели. Это заставляет модель быть проще и обобщать лучше.

- **L1-регуляризация (Lasso Regression):** Добавляет к функции потерь сумму абсолютных значений коэффициентов модели (L1-норма).

$$J(\theta) + \lambda \sum_{j=1}^n |\theta_j|$$

Где λ — параметр регуляризации, контролирующий силу штрафа. L1-регуляризация имеет свойство **отбора признаков**, так как она может обнулять коэффициенты менее важных признаков, фактически исключая их из модели.

- **L2-регуляризация (Ridge Regression):** Добавляет к функции потерь сумму квадратов значений коэффициентов модели (L2-норма).

$$J(\theta) + \lambda \sum_{j=1}^n \theta_j^2$$

L2-регуляризация стремится **уменьшить** значения коэффициентов, но редко обнуляет их полностью. Она распределяет влияние между всеми признаками.

Эластичная сеть (Elastic Net): Комбинирует L1 и L2 регуляризацию.

Вопрос 5: Логистическая регрессия: математическая модель, функция потерь, области применения.

Логистическая регрессия (Logistic Regression): Несмотря на название «регрессия», это алгоритм **классификации**, используемый для предсказания вероятности принадлежности объекта к одному из двух классов (бинарная классификация). Если вероятность превышает определенный порог (обычно 0.5), объект относится к одному классу, иначе — к другому.

Математическая модель: В основе логистической регрессии лежит **сигмоидная функция** (или логистическая функция), которая преобразует любое вещественное число в значение от 0 до 1, что можно интерпретировать как вероятность.

1. Сначала вычисляется линейная комбинация входных признаков, как в линейной регрессии: $z = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n = \theta^T x$

2. Затем это значение z передается через сигмоидную функцию: $P(y = 1|x) = \sigma(z) = \frac{1}{1+e^{-z}}$

Где $P(y = 1|x)$ — вероятность того, что объект x принадлежит к классу 1. Вероятность принадлежности к классу 0 будет $P(y = 0|x) = 1 - P(y = 1|x)$.

Функция потерь (Cost Function): Для логистической регрессии используется функция потерь, основанная на **кросс-энтропии** (или логарифмической функции потерь), так как метод наименьших квадратов не подходит из-за нелинейности сигмоидной функции, что приводит к невыпуклой функции потерь с множеством локальных минимумов.

Функция потерь для одного примера:

$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

Общая функция потерь для всего набора данных (которую нужно минимизировать):

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Минимизация этой функции потерь обычно выполняется с помощью градиентного спуска.

Области применения:

- **Медицина:** предсказание наличия заболевания (да/нет).
- **Финансы:** оценка кредитного риска (заемщик вернет/не вернет кредит).
- **Маркетинг:** предсказание оттока клиентов (уйдет/останется).
- **Спам-фильтры:** классификация писем как спам/не спам.
- **Оценка настроения:** определение позитивного/негативного тона текста.

Вопрос 6: Метрики классификации (Accuracy, Precision, Recall, F1-score, ROC-AUC). Матрица ошибок.

Матрица ошибок (Confusion Matrix): Это таблица, которая используется для оценки производительности алгоритма классификации, особенно когда есть два или более класса. Она показывает количество правильных и неправильных предсказаний, сделанных моделью, по сравнению с фактическими значениями.

	Предсказано Positive	Предсказано Negative
Фактически Positive	True Positive (TP)	False Negative (FN)
Фактически Negative	False Positive (FP)	True Negative (TN)

- **True Positive (TP):** Модель правильно предсказала положительный класс.
- **True Negative (TN):** Модель правильно предсказала отрицательный класс.
- **False Positive (FP):** Модель ошибочно предсказала положительный класс (ошибка I рода, ложная тревога).
- **False Negative (FN):** Модель ошибочно предсказала отрицательный класс (ошибка II рода, пропуск).

Метрики классификации:

- **Accuracy (Точность):** Доля правильно классифицированных объектов от общего числа объектов.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Хороша для сбалансированных классов, но может быть обманчива при несбалансированных данных (например, если 99% данных относятся к одному классу, модель, которая всегда предсказывает этот класс, будет иметь 99% Accuracy).

- **Precision (Точность предсказания положительного класса):** Доля правильно предсказанных положительных объектов среди всех объектов, которые модель предсказала как положительные. Отвечает на вопрос: «Из всех, кого мы назвали положительными, сколько действительно положительных?»

$$Precision = \frac{TP}{TP+FP}$$

Важна, когда стоимость ложноположительных срабатываний высока (например, ошибочное обвинение невиновного).

- **Recall (Полнота, Чувствительность):** Доля правильно предсказанных положительных объектов среди всех фактически положительных объектов. Отвечает на вопрос: «Из всех действительно положительных объектов, сколько мы смогли найти?»

$$Recall = \frac{TP}{TP+FN}$$

Важна, когда стоимость ложноотрицательных срабатываний высока (например, пропуск больного человека).

- **F1-score (F-мера):** Гармоническое среднее Precision и Recall. Используется, когда нужно найти баланс между Precision и Recall, особенно при несбалансированных классах.

$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- **ROC-AUC (Receiver Operating Characteristic - Area Under the Curve):** ROC-кривая — это график, который показывает производительность модели классификации при различных порогах классификации. Она строится путем откладывания True Positive Rate (Recall) по оси Y и False Positive Rate (FP / (FP + TN)) по оси X.

AUC (Area Under the Curve) — это площадь под ROC-кривой. Значение AUC варьируется от 0 до 1.

- $AUC = 1$: идеальный классификатор.
- $AUC = 0.5$: классификатор работает не лучше случайного угадывания.
- $AUC < 0.5$: классификатор хуже случайного угадывания (возможно, предсказывает наоборот).

ROC-AUC является хорошей метрикой для оценки моделей на несбалансированных данных, так как она не зависит от выбора порога классификации.

Вопрос 7: Метод опорных векторов (SVM): идея, ядровой трюк, области применения.

Метод опорных векторов (Support Vector Machine, SVM): Это мощный алгоритм классификации (и регрессии), который ищет оптимальную разделяющую гиперплоскость между классами в многомерном пространстве. Цель SVM — найти такую гиперплоскость, которая имеет максимально возможный отступ (margin) от ближайших точек каждого класса (эти точки называются **опорными векторами**).

Идея:

1. **Разделяющая гиперплоскость:** В простейшем случае (линейно разделимые данные) SVM находит прямую (или плоскость/гиперплоскость), которая наилучшим образом разделяет данные на два класса.
2. **Максимизация отступа:** Вместо того чтобы просто найти любую разделяющую гиперплоскость, SVM ищет ту, которая максимально удалена от ближайших точек каждого класса. Это делает модель более устойчивой к новым данным и улучшает обобщающую способность.
3. **Опорные векторы:** Точки данных, которые находятся ближе всего к разделяющей гиперплоскости и определяют ее положение, называются опорными векторами. Только эти точки важны для определения гиперплоскости; остальные данные можно удалить без изменения модели.

Ядровой трюк (Kernel Trick): Что делать, если данные не являются линейно разделимыми в исходном пространстве? SVM использует **ядровой трюк**. Идея

заключается в том, чтобы неявно преобразовать данные в пространство более высокой размерности, где они становятся линейно разделимыми, без явного вычисления координат этих точек в новом пространстве.

Ядровая функция (kernel function) вычисляет скалярное произведение векторов в пространстве высокой размерности, используя только исходные координаты. Это позволяет SVM эффективно работать с нелинейно разделимыми данными.

Популярные ядровые функции:

- **Линейное ядро (Linear Kernel):** Для линейно разделимых данных.
- **Полиномиальное ядро (Polynomial Kernel):** Для нелинейных зависимостей.
- **Радиально-базисная функция (Radial Basis Function, RBF) или Гауссово ядро:** Очень популярно, хорошо работает со сложными нелинейными зависимостями.

Области применения:

- **Распознавание образов:** распознавание рукописных цифр, лиц.
- **Биоинформатика:** классификация белков, анализ микрочипов.
- **Классификация текстов:** категоризация документов, спам-фильтры.
- **Медицинская диагностика:** классификация опухолей.

Вопрос 8: Деревья решений: идея, критерии ветвления (энтропия, индекс Джини), обрезка деревьев.

Деревья решений (Decision Trees): Это алгоритм машинного обучения, который используется как для классификации, так и для регрессии. Он строит модель в виде дерева, где каждый внутренний узел представляет собой проверку какого-либо признака, каждая ветвь — результат этой проверки, а каждый лиственный узел — предсказанное значение (класс или число).

Идея: Дерево решений последовательно разбивает набор данных на подмножества на основе значений признаков. Цель — создать такие разбиения, чтобы в каждом конечном (листовом) узле находились максимально однородные по целевой переменной объекты. Процесс построения дерева начинается с корневого узла и продолжается рекурсивно.

Критерии ветвления (Splitting Criteria): Для выбора наилучшего признака и порога для разбиения в каждом узле используются специальные критерии, которые измеряют «чистоту» или «однородность» подмножеств данных после разбиения. Чем больше уменьшается неопределенность (или увеличивается однородность), тем лучше разбиение.

- **Энтропия (Entropy):** Мера неопределенности или хаоса в наборе данных. Чем выше энтропия, тем более разнообразны классы в узле. Цель алгоритма — минимизировать энтропию после разбиения.

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

Где p_i — доля объектов класса i в наборе S .

Прирост информации (Information Gain): Разница между энтропией до разбиения и взвешенной суммой энтропий после разбиения. Алгоритм выбирает разбиение, которое максимизирует прирост информации.

- **Индекс Джини (Gini Impurity):** Мера вероятности того, что случайно выбранный элемент из набора будет неправильно классифицирован, если его класс выбирается случайным образом из распределения классов в наборе. Чем ниже индекс Джини, тем чище узел.

$$Gini(S) = 1 - \sum_{i=1}^c p_i^2$$

Алгоритм выбирает разбиение, которое минимизирует индекс Джини (или максимизирует уменьшение индекса Джини).

Обрезка деревьев (Pruning): Деревья решений склонны к переобучению, особенно если им позволить расти до тех пор, пока каждый листовой узел не будет содержать только один объект или объекты одного класса. Обрезка — это метод, используемый для борьбы с переобучением путем удаления ветвей, которые имеют малое значение для предсказательной способности модели.

Виды обрезки:

- **Предварительная обрезка (Pre-pruning):** Остановка роста дерева до того, как оно полностью разовьется. Это делается путем установки ограничений, таких как максимальная глубина дерева, минимальное количество объектов в узле для разбиения, минимальное количество объектов в листовом узле.

- **Пост-обрезка (Post-pruning):** Построение полного дерева, а затем удаление наименее значимых ветвей, обычно на основе оценки производительности на валидационном наборе данных.

Вопрос 9: Ансамблевые методы: бэггинг (случайный лес), бустинг (AdaBoost, градиентный бустинг, XGBoost).

Ансамблевые методы (Ensemble Methods): Это методы машинного обучения, которые объединяют предсказания нескольких базовых моделей (так называемых «слабых учеников») для получения более точного и устойчивого итогового предсказания. Идея в том, что коллективное решение нескольких моделей часто лучше, чем решение одной модели.

Бэггинг (Bagging - Bootstrap Aggregating): Основная идея бэггинга — уменьшить дисперсию (variance) модели, обучая несколько независимых моделей на разных подвыборках обучающих данных, а затем усредняя их предсказания (для регрессии) или голосуя (для классификации).

- **Алгоритм:**

1. Из исходного обучающего набора данных с помощью **бутстрэпа** (выборка с возвращением) создается несколько новых подвыборок.
2. На каждой подвыборке обучается независимая базовая модель (например, дерево решений).
3. Итоговое предсказание получается путем усреднения предсказаний всех базовых моделей (для регрессии) или голосования большинства (для классификации).

- **Случайный лес (Random Forest):** Это один из самых популярных ансамблевых методов, основанный на бэггинге, который использует деревья решений в качестве базовых моделей.

Ключевые особенности Случайного леса:

- **Случайность в данных:** Каждое дерево обучается на бутстрэп-выборке данных.
- **Случайность в признаках:** При выборе лучшего разбиения в каждом узле дерева рассматривается только случайное подмножество

признаков, а не все признаки. Это дополнительно уменьшает корреляцию между деревьями и снижает переобучение.

Случайный лес очень устойчив к переобучению, хорошо работает с большим количеством признаков и может оценивать важность признаков.

Бустинг (Boosting): В отличие от бэггинга, где модели обучаются независимо, бустинг строит ансамбль последовательно. Каждая новая базовая модель обучается для исправления ошибок, допущенных предыдущими моделями. Таким образом, бустинг фокусируется на «трудных» примерах, которые были неправильно классифицированы.

- **AdaBoost (Adaptive Boosting):** Один из первых и наиболее известных алгоритмов бустинга.

Идея:

1. Начинает с равных весов для всех обучающих примеров.
 2. Обучает базовую модель (обычно «слабый» классификатор, например, неглубокое дерево решений).
 3. Увеличивает веса тех примеров, которые были неправильно классифицированы, и уменьшает веса правильно классифицированных примеров.
 4. Следующая базовая модель обучается с учетом этих обновленных весов, уделяя больше внимания «трудным» примерам.
 5. Итоговое предсказание — это взвешенная сумма предсказаний всех базовых моделей, где веса моделей зависят от их точности.
- **Градиентный бустинг (Gradient Boosting):** Более общий и мощный подход. Вместо того чтобы изменять веса данных, как в AdaBoost, градиентный бустинг строит новые модели, которые предсказывают **остатки (residuals)** или **градиенты** ошибок предыдущих моделей. То есть каждая новая модель пытается исправить ошибки предыдущего ансамбля.

Идея:

1. Начинает с простой базовой модели (например, константы).
2. Последовательно добавляет новые модели, каждая из которых обучается на ошибках (градиентах) текущего ансамбля.

3. Каждая новая модель «толкает» ансамбль в направлении, которое уменьшает общую функцию потерь.

- **XGBoost (eXtreme Gradient Boosting):** Это оптимизированная и масштабируемая реализация градиентного бустинга, которая стала очень популярной благодаря своей высокой производительности и эффективности. XGBoost включает в себя множество улучшений:
 - **Регуляризация:** Включает L1 и L2 регуляризацию для предотвращения переобучения.
 - **Обработка пропущенных значений:** Может автоматически обрабатывать пропущенные значения.
 - **Параллельные вычисления:** Оптимизирован для параллельного выполнения.
 - **Обрезка деревьев:** Использует стратегию обрезки, которая улучшает обобщающую способность.

XGBoost часто показывает лучшие результаты на табличных данных и широко используется в соревнованиях по машинному обучению.

Вопрос 10: Кластеризация: K-Means, DBSCAN. Метрики качества кластеризации (Silhouette Score).

Кластеризация (Clustering): Это задача обучения без учителя, целью которой является группировка объектов в подмножества (кластеры) таким образом, чтобы объекты в одном кластере были более похожи друг на друга, чем на объекты в других кластерах. Кластеризация используется для поиска скрытых структур в данных.

K-Means: Один из самых популярных и простых алгоритмов кластеризации.

Идея:

1. **Инициализация:** Случайным образом выбираются K центроидов (центров кластеров) из данных.
2. **Присвоение:** Каждый объект данных присваивается к ближайшему центроиду (на основе евклидова расстояния).

3. **Обновление:** Центроиды пересчитываются как среднее арифметическое всех объектов, принадлежащих к их кластеру.
4. **Повторение:** Шаги 2 и 3 повторяются до тех пор, пока центроиды не перестанут значительно меняться или не будет достигнуто максимальное количество итераций.

Особенности K-Means:

- Требуется заранее задать количество кластеров K .
- Чувствителен к начальному выбору центроидов (может сходиться к локальным минимумам).
- Предполагает, что кластеры имеют сферическую форму и примерно одинаковый размер.
- Чувствителен к выбросам.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise): Алгоритм кластеризации, основанный на плотности. Он способен находить кластеры произвольной формы и хорошо справляется с шумом (выбросами).

Идея: DBSCAN определяет кластеры как области высокой плотности, разделенные областями низкой плотности. Он использует два основных параметра:

- **eps (epsilon):** Максимальное расстояние между двумя точками, чтобы одна считалась находящейся в окрестности другой.
- **min_samples :** Минимальное количество точек в окрестности (включая саму точку), чтобы точка считалась центральной (core point).

Типы точек:

- **Центральная точка (Core Point):** Точка, у которой в радиусе `eps` находится не менее `min_samples` других точек.
- **Граничная точка (Border Point):** Точка, которая находится в радиусе `eps` от центральной точки, но сама не является центральной (у нее меньше `min_samples` соседей).
- **Шумовая точка (Noise Point):** Точка, которая не является ни центральной, ни граничной.

Алгоритм:

1. Начинает с произвольной, непосещенной точки.
2. Если точка является центральной, формируется новый кластер, и все достижимые из нее точки (включая другие центральные и граничные) добавляются в этот кластер.
3. Процесс продолжается до тех пор, пока все точки не будут посещены.

Особенности DBSCAN:

- Не требует заранее задавать количество кластеров.
- Может находить кластеры произвольной формы.
- Эффективно обнаруживает выбросы (шумовые точки).
- Чувствителен к выбору параметров `eps` и `min_samples`.

Метрики качества кластеризации:

- **Silhouette Score (Коэффициент силуэта):** Метрика, используемая для оценки качества кластеризации, когда истинные метки классов неизвестны (в отличие от метрик классификации). Она измеряет, насколько объект похож на свой собственный кластер по сравнению с другими кластерами.

Значение Silhouette Score для одного объекта i вычисляется как:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Где:

- $a(i)$ — среднее расстояние от объекта i до всех других объектов в том же кластере (мера сплоченности).
- $b(i)$ — минимальное среднее расстояние от объекта i до объектов в другом кластере (мера разделенности).

Значение Silhouette Score варьируется от -1 до 1:

- **Близко к 1:** Объект хорошо соответствует своему кластеру и плохо соответствует соседним кластерам (хорошая кластеризация).
- **Близко к 0:** Объект находится близко к границе между двумя кластерами (может быть отнесен к любому из них).

- **Близко к -1:** Объект, вероятно, был отнесен не к тому кластеру.

Общий Silhouette Score для всего набора данных — это среднее значение Silhouette Score для всех объектов. Чем выше средний Silhouette Score, тем лучше качество кластеризации.

Вопрос 11: Снижение размерности: PCA (метод главных компонент), t-SNE.

Снижение размерности (Dimensionality Reduction): Это процесс уменьшения количества случайных переменных (признаков) в наборе данных путем получения набора главных переменных. Цель — упростить данные, уменьшить вычислительную сложность, бороться с «проклятием размерности» и улучшить визуализацию, сохраняя при этом как можно больше важной информации.

PCA (Principal Component Analysis - Метод главных компонент): Линейный метод снижения размерности, который преобразует набор, возможно, коррелированных переменных в набор линейно некоррелированных переменных, называемых **главными компонентами**.

Идея:

1. **Поиск направлений максимальной дисперсии:** PCA находит направления (векторы) в пространстве данных, вдоль которых данные имеют наибольшую дисперсию (разброс). Эти направления называются главными компонентами.
2. **Проекция:** Данные проецируются на эти новые оси. Первая главная компонента объясняет наибольшую долю дисперсии, вторая — наибольшую долю оставшейся дисперсии, и так далее.
3. **Уменьшение размерности:** Выбирается k главных компонент (где k значительно меньше исходного количества признаков), которые объясняют достаточную долю общей дисперсии, и данные представляются только этими k компонентами.

Особенности PCA:

- Линейный метод.
- Сохраняет глобальную структуру данных (расстояния между точками).

- Чувствителен к масштабу признаков (рекомендуется стандартизация данных перед применением PCA).
- Главные компоненты ортогональны друг другу.

t-SNE (t-Distributed Stochastic Neighbor Embedding): Нелинейный метод снижения размерности, который особенно хорошо подходит для визуализации высокоразмерных данных, проецируя их в двух- или трехмерное пространство. t-SNE стремится сохранить локальную структуру данных, то есть точки, которые были близки в высокоразмерном пространстве, остаются близкими и в низкоразмерном.

Идея:

1. **Моделирование сходства:** t-SNE строит распределение вероятностей, которое измеряет сходство между парами точек в высокоразмерном пространстве (используя Гауссово распределение).
2. **Моделирование сходства в низкоразмерном пространстве:** Затем оно строит аналогичное распределение вероятностей для точек в низкоразмерном пространстве (используя t-распределение Стюдента).
3. **Минимизация расхождения:** Алгоритм итеративно перемещает точки в низкоразмерном пространстве, чтобы минимизировать расхождение между этими двумя распределениями вероятностей (обычно с помощью дивергенции Кульбака-Лейблера). Это означает, что t-SNE пытается сделать так, чтобы «соседи» в высокоразмерном пространстве оставались «соседями» в низкоразмерном.

Особенности t-SNE:

- Нелинейный метод.
- Отлично подходит для визуализации, выявляя кластеры и скрытые структуры.
- Сохраняет локальную структуру данных, но может искажать глобальную.
- Вычислительно более затратен, чем PCA, особенно для очень больших наборов данных.
- Имеет гиперпараметр `perplexity`, который влияет на баланс между сохранением локальной и глобальной структуры.

Вопрос 12: Оценка качества моделей: кросс-валидация (K-Fold), Bootstrap.

Оценка качества моделей: После обучения модели машинного обучения необходимо оценить ее производительность, чтобы понять, насколько хорошо она обобщает на новые, невидимые данные. Простое тестирование на тех же данных, на которых модель обучалась, приведет к переоценке ее качества из-за переобучения. Поэтому используются методы, которые позволяют получить более надежную оценку.

Кросс-валидация (Cross-Validation): Это набор методов, которые позволяют оценить производительность модели на ограниченном наборе данных, используя все данные как для обучения, так и для тестирования, но в разных комбинациях. Это помогает получить более надежную оценку и уменьшить влияние случайного разделения данных.

- **K-Fold кросс-валидация:** Один из наиболее распространенных методов кросс-валидации.

Алгоритм:

1. Исходный набор данных случайным образом разбивается на K примерно равных по размеру подмножеств (фолдов).
2. Процесс повторяется K раз:
 - Один фолд используется как **тестовый набор**.
 - Остальные $K - 1$ фолдов объединяются и используются как **обучающий набор**.
 - Модель обучается на обучающем наборе и оценивается на тестовом наборе.
3. В конце усредняются метрики производительности, полученные на каждом из K тестовых наборов. Это среднее значение является оценкой производительности модели.

Преимущества K-Fold:

- Каждый объект данных используется ровно один раз в тестовом наборе и $K - 1$ раз в обучающем наборе.

- Дает более надежную оценку производительности модели, чем однократное разделение на обучение/тест.
- Помогает выявить проблемы переобучения или недообучения.

Выбор K :

- Обычно K выбирают равным 5 или 10.
 - При $K = N$ (где N — количество объектов) это называется **Leave-One-Out Cross-Validation (LOOCV)**, что очень вычислительно затратно.
- **Bootstrap:** Это метод повторной выборки, который используется для оценки статистических характеристик выборки (например, среднего, медианы, стандартного отклонения) или для оценки производительности модели.

Идея:

1. Из исходного набора данных многократно (например, 1000 раз) формируются новые выборки того же размера путем **выборки с возвращением** (то есть один и тот же объект может быть выбран несколько раз).
2. На каждой такой бутстрэп-выборке обучается модель.
3. Оценка производительности модели может быть получена путем усреднения результатов на всех бутстрэп-выборках. Также можно использовать объекты, которые не попали в бутстрэп-выборку (out-of-bag samples), для тестирования модели.

Преимущества Bootstrap:

- Позволяет оценить неопределенность в оценках (например, построить доверительные интервалы).
- Может быть полезен для небольших наборов данных.

Отличие от кросс-валидации: Кросс-валидация делит данные на непересекающиеся подмножества, тогда как бутстрэп создает выборки с возвращением, что означает, что некоторые объекты могут повторяться, а некоторые могут не попасть в выборку.

Вопрос 13: Метрики регрессии (MAE, MSE, RMSE, R2).

Метрики регрессии используются для оценки того, насколько хорошо модель предсказывает непрерывные числовые значения. Они измеряют разницу между предсказанными значениями и фактическими значениями.

Пусть y_i — фактическое значение, а $\hat{y}_i$ — предсказанное значение для i -го объекта.

- **MAE (Mean Absolute Error - Средняя абсолютная ошибка):** Среднее арифметическое абсолютных значений ошибок. Она измеряет среднюю величину ошибок в предсказаниях, не учитывая их направление.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Особенности:

- Легко интерпретируется: показывает среднюю абсолютную разницу между предсказаниями и фактическими значениями.
 - Менее чувствительна к выбросам, чем MSE, так как не возводит ошибки в квадрат.
- **MSE (Mean Squared Error - Средняя квадратичная ошибка):** Среднее арифметическое квадратов ошибок. Она штрафует большие ошибки сильнее, чем маленькие, так как ошибки возводятся в квадрат.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Особенности:

- Более чувствительна к выбросам, чем MAE.
 - Используется во многих алгоритмах как функция потерь (например, в линейной регрессии).
 - Единицы измерения MSE — это квадрат единиц измерения целевой переменной, что может затруднять интерпретацию.
- **RMSE (Root Mean Squared Error - Корень из средней квадратичной ошибки):** Квадратный корень из MSE. Возвращает метрику в тех же единицах измерения, что и целевая переменная, что делает ее более интерпретируемой, чем MSE.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Особенности:

- Как и MSE, сильно штрафует большие ошибки.
- Хорошо интерпретируется, так как имеет те же единицы, что и целевая переменная.
- **R2 (R-squared - Коэффициент детерминации):** Показывает долю дисперсии зависимой переменной, которая объясняется моделью. Значение R2 варьируется от 0 до 1 (хотя может быть и отрицательным для очень плохих моделей).

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Где $\bar{y}$ — среднее значение фактической целевой переменной.

Особенности:

- **R2 = 1:** Модель идеально объясняет всю дисперсию целевой переменной.
- **R2 = 0:** Модель не объясняет никакой дисперсии (предсказывает не лучше, чем просто среднее значение).
- **R2 < 0:** Модель работает хуже, чем простое предсказание среднего значения.
- R2 может увеличиваться при добавлении новых признаков, даже если они не улучшают модель. Для учета этого существует **скорректированный R2 (Adjusted R-squared)**.

Вопрос 14: Временные ряды: основные понятия, компоненты временного ряда (тренд, сезонность, цикличность, шум), методы прогнозирования (ARIMA, Prophet).

Временные ряды (Time Series): Это последовательность точек данных, индексированных (или перечисленных) в хронологическом порядке. Данные временных рядов собираются через последовательные интервалы времени. Примеры: ежедневные цены акций, ежемесячные продажи, ежечасная температура.

Основные понятия:

- **Стационарность:** Временной ряд считается стационарным, если его статистические свойства (среднее, дисперсия, автокорреляция) не меняются со временем. Многие модели временных рядов предполагают стационарность.
- **Автокорреляция:** Зависимость между значениями временного ряда в разные моменты времени. Например, сегодняшняя цена акции может зависеть от вчерашней.

Компоненты временного ряда: Временной ряд часто можно разложить на несколько составляющих:

- **Тренд (Trend):** Долгосрочное увеличение или уменьшение значений ряда. Это общая направленность данных с течением времени. Например, рост населения или увеличение продаж продукта на протяжении нескольких лет.
- **Сезонность (Seasonality):** Регулярные, предсказуемые колебания, которые повторяются через фиксированные интервалы времени (например, ежедневно, еженедельно, ежемесячно, ежегодно). Примеры: увеличение продаж мороженого летом, рост трафика на сайте в будние дни.
- **Цикличность (Cyclicity):** Колебания, которые не имеют фиксированного периода и обычно длятся дольше, чем сезонные колебания (например, экономические циклы, циклы деловой активности). Они не так регулярны, как сезонность.
- **Шум (Noise) / Остаток (Residual):** Случайные, непредсказуемые колебания, которые остаются после удаления тренда, сезонности и цикличности. Это то, что модель не смогла объяснить.

Методы прогнозирования:

- **ARIMA (AutoRegressive Integrated Moving Average):** Это один из классических и наиболее широко используемых статистических методов для прогнозирования временных рядов. Модель ARIMA состоит из трех частей:
 - **AR (AutoRegressive):** Использует прошлые значения ряда для предсказания будущих. Параметр p указывает количество прошлых

наблюдений, используемых в модели.

- **I (Integrated):** Использует дифференцирование (вычисление разностей между последовательными наблюдениями) для того, чтобы сделать временной ряд стационарным. Параметр d указывает количество раз, которое нужно дифференцировать ряд.
- **MA (Moving Average):** Использует прошлые ошибки прогнозирования для предсказания будущих. Параметр q указывает количество прошлых ошибок, используемых в модели.

Модель обозначается как $ARIMA(p, d, q)$.

Особенности ARIMA:

- Требуется стационарности ряда (или его приведения к стационарному виду).
 - Хорошо работает для одномерных временных рядов.
 - Требуется экспертного подбора параметров p, d, q (например, с помощью анализа автокорреляционной и частной автокорреляционной функций).
- **Prophet:** Библиотека с открытым исходным кодом от Facebook, разработанная для прогнозирования временных рядов, особенно хорошо подходящая для данных с сильной сезонностью и наличием праздников. Prophet более прост в использовании для неспециалистов, чем ARIMA.

Идея: Prophet разлагает временной ряд на компоненты: тренд, сезонность (годовая, недельная, дневная), влияние праздников и остаток. Он использует аддитивную модель:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

Где:

- $g(t)$ — функция тренда (нелинейный рост или спад).
- $s(t)$ — периодические изменения (сезонность).
- $h(t)$ — влияние праздников.
- ϵ_t — остаточный шум.

Особенности Prophet:

- Хорошо обрабатывает пропущенные данные и выбросы.
- Автоматически учитывает сезонность и праздники.
- Интуитивно понятен и прост в настройке.
- Подходит для прогнозирования на больших масштабах.
- Менее чувствителен к стационарности ряда, чем ARIMA.

Вопрос 15: Нейронные сети: перцептрон, многослойный перцептрон (MLP), функции активации (ReLU, Sigmoid, Tanh), обратное распространение ошибки.

Нейронные сети (Neural Networks): Это вычислительные модели, вдохновленные структурой и функционированием человеческого мозга. Они состоят из взаимосвязанных узлов (нейронов), организованных в слои, которые обрабатывают информацию и учатся на данных.

Перцептрон (Perceptron): Самая простая форма нейронной сети, разработанная Фрэнком Розенблаттом. Это линейный классификатор, который может разделять данные на два класса, если они линейно разделимы.

Идея:

1. **Взвешенная сумма:** Перцептрон принимает несколько входных сигналов, умножает каждый на соответствующий вес, а затем суммирует их.
2. **Функция активации:** К этой сумме применяется функция активации (обычно пороговая функция), которая выдает 1, если сумма превышает порог, и 0 (или -1) в противном случае.
3. **Обучение:** Перцептрон обучается путем корректировки весов, чтобы уменьшить ошибки предсказания. Если предсказание неверно, веса изменяются в направлении, которое уменьшит ошибку.

Ограничения: Перцептрон может решать только линейно разделимые задачи (например, логические функции И, ИЛИ), но не может решить задачу XOR.

Многослойный перцептрон (Multi-Layer Perceptron, MLP): Это более сложная нейронная сеть, состоящая из как минимум трех слоев нейронов:

1. **Входной слой (Input Layer):** Принимает входные данные.

2. **Один или несколько скрытых слоев (Hidden Layers):** Выполняют основные вычисления и извлечение признаков. Каждый нейрон в скрытом слое связан со всеми нейронами предыдущего и следующего слоя.
3. **Выходной слой (Output Layer):** Выдает окончательные предсказания.

Нейроны в MLP используют нелинейные функции активации, что позволяет MLP решать нелинейные задачи и аппроксимировать любую непрерывную функцию (теорема универсальной аппроксимации).

Функции активации (Activation Functions): Это нелинейные функции, которые применяются к взвешенной сумме входов нейрона. Они вводят нелинейность в модель, что позволяет нейронным сетям учиться сложным закономерностям.

- **Сигмоидная функция (Sigmoid):** $\sigma(x) = \frac{1}{1+e^{-x}}$

Выход находится в диапазоне от 0 до 1. Часто используется в выходном слое для задач бинарной классификации (как вероятность). **Проблема:** Страдает от проблемы «исчезающего градиента» (vanishing gradient) для очень больших или очень маленьких входных значений, что замедляет обучение глубоких сетей.

- **Гиперболический тангенс (Tanh - Hyperbolic Tangent):** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Выход находится в диапазоне от -1 до 1. Похожа на сигмоиду, но центрирована относительно нуля, что часто делает обучение более стабильным, чем с сигмоидой. Также страдает от проблемы исчезающего градиента.

- **ReLU (Rectified Linear Unit):** $ReLU(x) = \max(0, x)$

Выход равен x , если $x > 0$, и 0, если $x \leq 0$. **Преимущества:** Решает проблему исчезающего градиента для положительных значений, вычислительно эффективна. **Проблема:** Может страдать от проблемы «умирающих ReLU» (dying ReLU), когда нейроны перестают активироваться и, следовательно, обучаться.

Обратное распространение ошибки (Backpropagation): Это основной алгоритм обучения многослойных нейронных сетей. Он используется для эффективного вычисления градиентов функции потерь по отношению ко всем весам в сети, что позволяет обновлять веса с помощью градиентного спуска.

Идея:

1. **Прямое распространение (Forward Pass):** Входные данные подаются на входной слой, проходят через скрытые слои, и сеть генерирует предсказание на выходном слое.
2. **Вычисление ошибки:** Сравнивается предсказание сети с фактическим целевым значением, и вычисляется ошибка (значение функции потерь).
3. **Обратное распространение (Backward Pass):** Ошибка распространяется назад через сеть, от выходного слоя к входному. На каждом слое вычисляется, насколько каждый вес и смещение способствовали общей ошибке (это и есть градиенты).
4. **Обновление весов:** Используя вычисленные градиенты, веса и смещения сети корректируются с помощью градиентного спуска, чтобы уменьшить ошибку.

Этот процесс повторяется многократно (эпохи) до тех пор, пока сеть не достигнет желаемой производительности или не перестанет улучшаться.

Вопрос 16: Сверточные нейронные сети (CNN): идея, сверточные слои, пулинг-слои, области применения.

Сверточные нейронные сети (Convolutional Neural Networks, CNN): Это специализированный тип нейронных сетей, который особенно эффективен для обработки данных, имеющих сеточную топологию, таких как изображения (2D-сетка пикселей), видео или аудио. CNN автоматически извлекают иерархические признаки из входных данных.

Идея: CNN используют концепцию **свертки** для обнаружения локальных паттернов в данных. Вместо того чтобы каждый нейрон был связан со всеми входами, как в MLP, нейроны в CNN связаны только с небольшой областью входных данных. Это позволяет сети учиться иерархическим признакам: на низких уровнях — края и углы, на более высоких — части объектов, а затем и целые объекты.

Основные строительные блоки CNN:

- **Сверточные слои (Convolutional Layers):** Это сердце CNN. В сверточном слое используются **фильтры (ядра)** — небольшие матрицы весов, которые

«скользят» по входному изображению (или карте признаков из предыдущего слоя) и выполняют операцию свертки.

Операция свертки: Фильтр умножается поэлементно на небольшую область входных данных, и результаты суммируются, образуя один пиксель в выходной **карте признаков (feature map)**. Каждый фильтр предназначен для обнаружения определенного паттерна (например, горизонтальных линий, вертикальных линий, углов).

Параметры сверточного слоя:

- **Размер фильтра (Kernel Size):** Размеры матрицы фильтра (например, 3x3, 5x5).
 - **Количество фильтров:** Определяет количество различных паттернов, которые слой будет искать.
 - **Шаг (Stride):** Расстояние, на которое фильтр перемещается по входным данным.
 - **Заполнение (Padding):** Добавление нулей по краям входных данных для сохранения пространственного размера выходной карты признаков.
- **Пулинг-слои (Pooling Layers):** Эти слои используются для уменьшения пространственного размера карт признаков, что помогает уменьшить количество параметров и вычислительную нагрузку, а также сделать модель более устойчивой к небольшим изменениям во входных данных (инвариантность к сдвигу).

Типы пулинга:

- **Макс-пулинг (Max Pooling):** Выбирает максимальное значение из небольшой области входных данных.
- **Средний пулинг (Average Pooling):** Вычисляет среднее значение из небольшой области входных данных.

Макс-пулинг является наиболее распространенным.

Типичная архитектура CNN: Последовательность сверточных и пулинг-слоев, за которой следуют один или несколько полносвязных слоев (как в MLP) для выполнения окончательной классификации или регрессии.

Области применения:

- **Компьютерное зрение:** распознавание изображений, обнаружение объектов, сегментация изображений, распознавание лиц.
- **Обработка естественного языка:** классификация текстов, анализ настроений (хотя для этого чаще используются RNN/Transformer).
- **Медицинская визуализация:** анализ медицинских снимков (рентген, МРТ).

Вопрос 17: Рекуррентные нейронные сети (RNN): идея, проблема исчезающего/взрывающегося градиента, LSTM, области применения.

Рекуррентные нейронные сети (Recurrent Neural Networks, RNN): Это тип нейронных сетей, разработанный для обработки последовательных данных, таких как текст, речь, временные ряды. В отличие от традиционных нейронных сетей, RNN имеют «память» — они могут использовать информацию из предыдущих шагов последовательности для обработки текущего шага.

Идея: Нейрон в RNN получает не только текущий вход, но и выход (или скрытое состояние) из предыдущего шага времени. Это позволяет RNN сохранять информацию о прошлых элементах последовательности и использовать ее для принятия решений на текущем шаге.

Проблема исчезающего/взрывающегося градиента (Vanishing/Exploding Gradient Problem): При обучении глубоких нейронных сетей (включая RNN) с помощью обратного распространения ошибки градиенты могут стать либо очень маленькими (исчезающий градиент), либо очень большими (взрывающийся градиент).

- **Исчезающий градиент:** Градиенты становятся настолько малыми, что обновления весов становятся незначительными, и сеть перестает эффективно обучаться, особенно для долгосрочных зависимостей (то есть, RNN забывает информацию, которая была далеко в прошлом последовательности).
- **Взрывающийся градиент:** Градиенты становятся настолько большими, что веса обновляются слишком сильно, что приводит к нестабильности обучения и расхождению модели.

LSTM (Long Short-Term Memory - Долгая краткосрочная память): Это особый тип рекуррентных нейронных сетей, разработанный для решения проблемы исчезающего градиента и эффективного обучения долгосрочным зависимостям. LSTM-ячейки имеют более сложную внутреннюю структуру, чем обычные RNN-нейроны, и включают в себя «вентили» (gates), которые контролируют поток информации.

Основные компоненты LSTM-ячейки:

- **Вентиль забывания (Forget Gate):** Решает, какую информацию из предыдущего состояния ячейки следует «забыть» (отбросить).
- **Вентиль ввода (Input Gate):** Решает, какую новую информацию из текущего входа следует «запомнить» и добавить в состояние ячейки.
- **Вентиль вывода (Output Gate):** Решает, какую часть текущего состояния ячейки следует вывести как скрытое состояние для следующего шага и для предсказания.

Эти вентили позволяют LSTM избирательно хранить и извлекать информацию на протяжении длительных последовательностей, эффективно решая проблему исчезающего градиента.

Области применения:

- **Обработка естественного языка (NLP):** машинный перевод, генерация текста, анализ тональности, распознавание именованных сущностей.
- **Распознавание речи:** преобразование аудио в текст.
- **Прогнозирование временных рядов:** предсказание цен акций, погодных условий.
- **Генерация музыки и видео.**

Вопрос 18: Трансформеры (Transformers): идея, механизм внимания (Self-Attention), области применения.

Трансформеры (Transformers): Это архитектура нейронных сетей, представленная в 2017 году, которая произвела революцию в области обработки естественного языка (NLP) и сейчас активно применяется в других областях, таких как компьютерное зрение. Ключевая особенность Трансформеров — это использование **механизма внимания (Attention Mechanism)**, который

позволяет модели взвешивать важность различных частей входной последовательности при обработке каждого элемента.

Идея: В отличие от RNN, которые обрабатывают последовательности пошагово, Трансформеры обрабатывают всю последовательность целиком, что позволяет им эффективно использовать параллельные вычисления и улавливать долгосрочные зависимости без проблем исчезающего градиента. Они полностью отказываются от рекуррентных и сверточных слоев в пользу механизма внимания.

Механизм внимания (Attention Mechanism): Это ключевой компонент Трансформеров. Он позволяет модели динамически определять, на какие части входной последовательности следует сосредоточиться при генерации выходного элемента.

- **Self-Attention (Механизм самовнимания):** Это особый тип механизма внимания, который позволяет модели взвешивать важность различных слов (или токенов) **внутри одной и той же входной последовательности** при обработке каждого слова.

Как это работает (упрощенно):

1. Для каждого слова в последовательности генерируются три вектора: **Query (Q), Key (K), Value (V)**. Эти векторы получаются путем умножения вектора представления слова на три разные матрицы весов.
2. Для каждого слова (Query) вычисляется его «схожесть» со всеми другими словами в последовательности (Key) с помощью скалярного произведения. Это дает **оценки внимания (attention scores)**.
3. Оценки внимания нормализуются (обычно с помощью Softmax), чтобы получить **веса внимания**, которые показывают, насколько сильно каждое другое слово должно влиять на текущее слово.
4. Векторы Value всех слов взвешиваются по этим весам внимания и суммируются. Результат — это новое представление слова, которое содержит информацию, «взвешенную» по важности других слов в контексте.

Это позволяет модели понимать контекст слова, учитывая все остальные слова в предложении, независимо от их расстояния друг от друга.

Архитектура Трансформера: Трансформер состоит из двух основных частей:

- **Кодировщик (Encoder):** Обрабатывает входную последовательность. Состоит из нескольких идентичных слоев, каждый из которых содержит механизм самовнимания и полносвязную нейронную сеть.
- **Декодировщик (Decoder):** Генерирует выходную последовательность. Также состоит из нескольких идентичных слоев, но включает дополнительный механизм внимания, который позволяет ему обращать внимание на выход кодировщика.

Области применения:

- **Обработка естественного языка (NLP):** машинный перевод (основное применение), генерация текста (GPT-3, GPT-4), вопросно-ответные системы, суммаризация текста, анализ настроений.
- **Компьютерное зрение:** Vision Transformers (ViT) показывают отличные результаты в задачах классификации изображений и обнаружения объектов.
- **Биоинформатика:** анализ последовательностей ДНК и белков.

Вопрос 19: Обучение с подкреплением: основные понятия (агент, среда, состояние, действие, награда, политика), Q-Learning.

Обучение с подкреплением (Reinforcement Learning, RL): Это область машинного обучения, где **агент** учится принимать решения в **среде** для достижения определенной цели. Агент не получает явных инструкций, а учится путем проб и ошибок, взаимодействуя со средой и получая **награды** или **штрафы** за свои действия.

Основные понятия:

- **Агент (Agent):** Сущность, которая принимает решения и взаимодействует со средой. Это может быть робот, программа, играющий в игру, или система управления.
- **Среда (Environment):** Мир, в котором действует агент. Среда получает действия от агента и возвращает ему новое **состояние** и **награду**.

- **Состояние (State, S):** Текущая ситуация, в которой находится агент. Состояние описывает все релевантные аспекты среды, которые агент может использовать для принятия решения.
- **Действие (Action, A):** Выбор, который агент делает в данном состоянии. Действие изменяет состояние среды.
- **Награда (Reward, R):** Числовое значение, которое агент получает от среды после выполнения действия. Положительная награда указывает на желаемое действие, отрицательная — на нежелательное. Цель агента — максимизировать суммарную награду за длительный период.
- **Политика (Policy, π):** Стратегия агента, которая определяет, какое действие выбрать в каждом состоянии. Политика может быть детерминированной (всегда одно и то же действие в данном состоянии) или стохастической (вероятностное распределение действий).

Q-Learning: Это один из самых популярных алгоритмов обучения с подкреплением без модели (model-free reinforcement learning). Он относится к классу алгоритмов **обучения ценности (value-based learning)**, где агент учится функции ценности, которая оценивает «хорошесть» выполнения определенного действия в определенном состоянии.

Идея: Q-Learning учится **Q-функции (Q-value function)**, обозначаемой как $Q(s, a)$. Эта функция возвращает ожидаемую максимальную будущую награду, которую агент получит, если он выполнит действие a в состоянии s , а затем будет следовать оптимальной политике.

Алгоритм (упрощенно):

1. **Инициализация Q-таблицы:** Создается таблица (Q-таблица), где строки соответствуют состояниям, а столбцы — действиям. Все значения Q-таблицы инициализируются нулями или случайными числами.
2. **Итерации:** Агент взаимодействует со средой в течение множества эпизодов:
 - В текущем состоянии s , агент выбирает действие a . Часто используется стратегия **ϵ -жадного выбора (ϵ -greedy)**: с вероятностью ϵ выбирается случайное действие (для исследования), а с вероятностью $1 - \epsilon$

выбирается действие с максимальным Q-значением (для эксплуатации).

- Агент выполняет действие a , получает награду R и переходит в новое состояние s' .
- Q-значение для пары (s, a) обновляется с использованием **уравнения Беллмана**:

$$Q(s, a) := Q(s, a) + \alpha[R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Где:

- α (скорость обучения): определяет, насколько сильно новое знание влияет на текущее Q-значение.
 - γ (фактор дисконтирования): определяет важность будущих наград. Значение от 0 до 1. Близкое к 0 означает, что агент заботится только о немедленных наградах; близкое к 1 означает, что он ценит долгосрочные награды.
 - $\max_{a'} Q(s', a')$: максимальное Q-значение для следующего состояния s' по всем возможным действиям a' .
- Состояние s обновляется до s' .

Особенности Q-Learning:

- **Безмодельный**: Не требует знания о том, как работает среда (функции перехода состояний и функции награды).
- **Off-policy**: Может учиться оптимальной политике, даже если агент следует другой (исследовательской) политикой.
- Хорошо подходит для задач с дискретными состояниями и действиями.

Вопрос 20: Рекомендательные системы: коллаборативная фильтрация (item-based, user-based), контентные рекомендации, гибридные системы.

Рекомендательные системы (Recommender Systems): Это системы, которые предсказывают предпочтения пользователя и предлагают ему товары, услуги или информацию, которые, скорее всего, ему понравятся. Они широко

используются в электронной коммерции, стриминговых сервисах, социальных сетях и т.д.

Типы рекомендательных систем:

- **Коллаборативная фильтрация (Collaborative Filtering):** Основана на идее, что люди, которые соглашались в прошлом, будут соглашаться и в будущем. Она ищет похожих пользователей или похожие элементы и делает рекомендации на основе их предпочтений.
 - **User-Based Collaborative Filtering (Пользовательская коллаборативная фильтрация):**
 - **Идея:** Найти пользователей, которые похожи на текущего пользователя (имеют схожие оценки или предпочтения).
 - **Как работает:** Если пользователь А и пользователь В имеют схожие вкусы (например, оба любят одни и те же фильмы), то фильм, который понравился пользователю В, но который пользователь А еще не видел, может быть рекомендован пользователю А.
 - **Проблема:** Масштабируемость для большого количества пользователей, проблема «холодного старта» для новых пользователей.
 - **Item-Based Collaborative Filtering (Предметная коллаборативная фильтрация):**
 - **Идея:** Найти элементы (товары, фильмы), которые похожи друг на друга (часто оцениваются одинаково или покупаются вместе).
 - **Как работает:** Если пользователь любит фильм Х, а фильм Х похож на фильм Y (потому что другие пользователи, которым понравился Х, также любили Y), то фильм Y может быть рекомендован пользователю.
 - **Преимущества:** Более масштабируема, чем user-based, так как схожесть между элементами меняется медленнее, чем между пользователями.
- **Контентные рекомендации (Content-Based Recommender Systems):** Основаны на характеристиках самих элементов и профиле предпочтений

пользователя.

- **Идея:** Если пользователь любит определенный тип контента в прошлом, ему будут рекомендованы похожие элементы.
 - **Как работает:** Создается профиль пользователя на основе его прошлых взаимодействий (например, жанры фильмов, ключевые слова в статьях, авторы). Затем система ищет новые элементы, чьи характеристики соответствуют профилю пользователя.
 - **Преимущества:** Не страдает от проблемы «холодного старта» для новых элементов (если у них есть описание), может рекомендовать уникальные элементы.
 - **Проблема:** Ограничена тем, что пользователь будет получать рекомендации только того, что уже ему нравилось, нет возможности открыть что-то новое (filter bubble).
- **Гибридные системы (Hybrid Recommender Systems):** Объединяют несколько подходов (например, коллаборативную фильтрацию и контентные рекомендации) для преодоления ограничений каждого из них и улучшения качества рекомендаций.
 - **Идея:** Использовать сильные стороны разных методов.
 - **Примеры:**
 - Объединение предсказаний разных моделей.
 - Использование контентных признаков для улучшения коллаборативной фильтрации.
 - Использование коллаборативной фильтрации для преодоления проблемы «холодного старта» для новых пользователей в контентных системах.
 - **Преимущества:** Часто дают лучшие результаты, чем любой отдельный метод, более устойчивы к проблемам «холодного старта» и разреженности данных.

Вопрос 21: Обработка естественного языка (NLP): основные задачи (токенизация, стемминг, лемматизация, POS-теггинг, NER), Word Embeddings (Word2Vec).

Обработка естественного языка (Natural Language Processing, NLP): Это область искусственного интеллекта, которая занимается взаимодействием компьютеров и человеческого (естественного) языка. Цель NLP — научить компьютеры понимать, интерпретировать и генерировать человеческий язык.

Основные задачи NLP:

- **Токенизация (Tokenization):** Процесс разбиения текста на более мелкие единицы, называемые **токенами**. Токенами могут быть слова, фразы, символы или даже целые предложения. Например, предложение «Привет, мир!» может быть токенизировано как [«Привет», «,» , «мир», «!»].
- **Стемминг (Stemming):** Процесс уменьшения слова до его базовой или корневой формы (стема) путем удаления суффиксов и префиксов. Стемма не обязательно является лингвистически правильным словом. Например, слова «бежать», «бежит», «бегущий» могут быть сведены к стему «беж». **Цель:** Уменьшить количество уникальных слов и обрабатывать разные формы одного слова как одно и то же.
- **Лемматизация (Lemmatization):** Более сложный процесс, чем стемминг, который также уменьшает слово до его базовой формы (леммы), но при этом гарантирует, что лемма является лингвистически правильным словом. Для этого используются словари и морфологический анализ. Например, слова «был», «есть», «будет» будут лемматизированы до «быть». **Цель:** Аналогична стеммингу, но с сохранением грамматической корректности.
- **POS-теггинг (Part-of-Speech Tagging - Морфологическая разметка):** Процесс присвоения каждому слову в предложении его грамматической категории (части речи), такой как существительное, глагол, прилагательное, наречие и т.д. Например, в предложении «Я люблю машинное обучение»: «Я» (местоимение), «люблю» (глагол), «машинное» (прилагательное), «обучение» (существительное). **Цель:** Помогает понять синтаксическую структуру предложения и значение слов.
- **NER (Named Entity Recognition - Извлечение именованных сущностей):** Задача по идентификации и классификации именованных сущностей в

тексте по predetermined категориям, таким как имена людей, названия организаций, географические местоположения, даты, время и т.д. Например, в предложении «Apple купила стартап в Лондоне в 2023 году»: «Apple» (Организация), «Лондоне» (Местоположение), «2023 году» (Дата). **Цель:** Извлечение структурированной информации из неструктурированного текста.

Word Embeddings (Векторные представления слов): Это методы представления слов в виде плотных векторов чисел (обычно с сотнями измерений), где слова с похожим значением имеют похожие векторные представления (близки в векторном пространстве). Это позволяет моделям машинного обучения работать со словами как с числовыми данными, улавливая семантические и синтаксические отношения между ними.

- **Word2Vec:** Один из самых известных и влиятельных алгоритмов для создания Word Embeddings, разработанный Google. Word2Vec использует неглубокую нейронную сеть для обучения векторных представлений слов.

Две основные архитектуры Word2Vec:

1. **CBOW (Continuous Bag-of-Words):** Предсказывает текущее слово на основе окружающих его слов (контекста).
2. **Skip-gram:** Предсказывает окружающие слова (контекст) на основе текущего слова.

Идея: Слова, которые часто встречаются вместе или в похожих контекстах, будут иметь похожие векторные представления. Например, векторы для «король» и «мужчина» будут похожи, как и для «королева» и «женщина», и даже можно наблюдать аналогии типа «король - мужчина + женщина = королева» в векторном пространстве.

Преимущества: Позволяет моделям понимать семантику слов, улучшает производительность во многих задачах NLP.

Вопрос 22: Компьютерное зрение: основные задачи (классификация изображений, обнаружение объектов, сегментация изображений), аугментация данных.

Компьютерное зрение (Computer Vision): Это область искусственного интеллекта, которая позволяет компьютерам «видеть» и интерпретировать цифровые изображения и видео. Цель — научить машины понимать визуальный мир так же, как это делает человек.

Основные задачи компьютерного зрения:

- **Классификация изображений (Image Classification):** Задача присвоения метки (класса) всему изображению. Модель определяет, что изображено на картинке. Например, на вход подается изображение кошки, и модель выдает «кошка». **Примеры:** Распознавание видов животных, классификация типов растений, определение категории товара на фотографии.
- **Обнаружение объектов (Object Detection):** Задача не только классификации объектов на изображении, но и определения их местоположения с помощью **ограничивающих рамок (bounding boxes)**. Модель отвечает на вопросы: «Что это?» и «Где это находится?» **Примеры:** Обнаружение пешеходов на дороге, поиск лиц на фотографии, идентификация различных объектов в сцене.
- **Сегментация изображений (Image Segmentation):** Более детальная задача, чем обнаружение объектов. Она включает в себя разделение изображения на пиксельные области, каждая из которых соответствует определенному объекту или классу. То есть, для каждого пикселя изображения модель предсказывает, к какому объекту он принадлежит.
 - **Семантическая сегментация:** Классифицирует каждый пиксель по классу объекта (например, «дорога», «небо», «человек»), но не различает отдельные экземпляры одного класса.
 - **Сегментация экземпляров:** Классифицирует каждый пиксель и различает отдельные экземпляры объектов (например, «человек 1», «человек 2»). **Примеры:** Автономное вождение (понимание границ дороги, других машин), медицинская визуализация (выделение опухолей), редактирование изображений (выделение фона).

Аугментация данных (Data Augmentation): Это набор методов, используемых для искусственного увеличения размера обучающего набора данных путем создания модифицированных версий существующих изображений. Это помогает улучшить обобщающую способность модели и снизить переобучение, особенно когда доступно ограниченное количество данных.

Типичные методы аугментации для изображений:

- **Геометрические преобразования:**
 - **Повороты (Rotation):** Поворот изображения на определенный угол.
 - **Отражения (Flipping):** Зеркальное отражение по горизонтали или вертикали.
 - **Масштабирование (Scaling):** Изменение размера изображения.
 - **Сдвиги (Translation):** Перемещение изображения по осям X или Y.
 - **Обрезка (Cropping):** Вырезание случайных частей изображения.
- **Изменения цвета:**
 - **Изменение яркости (Brightness Adjustment):** Осветление или затемнение.
 - **Изменение контрастности (Contrast Adjustment):** Увеличение или уменьшение контраста.
 - **Изменение насыщенности (Saturation Adjustment):** Изменение интенсивности цветов.
 - **Добавление шума (Adding Noise):** Добавление случайных пикселей.

Преимущества аугментации данных:

- **Увеличение размера данных:** Помогает обучать более сложные модели, когда исходных данных мало.
- **Снижение переобучения:** Модель становится более устойчивой к вариациям во входных данных.
- **Улучшение обобщающей способности:** Модель лучше работает на новых, невидимых данных.

Вопрос 23: Генеративные состязательные сети (GAN): идея, генератор, дискриминатор, области применения.

Генеративные состязательные сети (Generative Adversarial Networks, GAN):

Это класс алгоритмов машинного обучения, представленный Иэном Гудфеллоу в 2014 году. GAN состоят из двух нейронных сетей, которые «соревнуются» друг с другом в процессе обучения, что позволяет им генерировать новые данные, похожие на обучающие, но не идентичные им.

Идея: GAN можно представить как игру между двумя игроками:

- **Генератор (Generator, G):** Это нейронная сеть, которая принимает на вход случайный шум (вектор латентного пространства) и пытается сгенерировать новые данные (например, изображения), которые выглядят реалистично и похожи на реальные обучающие данные.
- **Дискриминатор (Discriminator, D):** Это нейронная сеть, которая принимает на вход либо реальные данные из обучающего набора, либо сгенерированные данные от Генератора. Его задача — отличить реальные данные от поддельных (сгенерированных).

Процесс обучения:

1. **Генератор** пытается обмануть **Дискриминатор**, создавая максимально реалистичные поддельные данные.
2. **Дискриминатор** пытается стать как можно лучше в распознавании поддельных данных от реальных.

Это состязание продолжается до тех пор, пока Генератор не станет настолько хорош, что Дискриминатор не сможет отличить его подделки от реальных данных (или сможет это делать только с вероятностью 50%). В этот момент Генератор способен создавать очень реалистичные новые данные.

Области применения:

- **Генерация реалистичных изображений:** создание лиц людей, которых не существует, пейзажей, объектов.
- **Увеличение разрешения изображений (Super-resolution):** улучшение качества изображений.

- **Перенос стиля (Style Transfer):** применение стиля одного изображения к другому.
- **Преобразование изображений:** например, из эскиза в фотографию, из дневного в ночное время.
- **Аугментация данных:** создание дополнительных обучающих примеров для других моделей.
- **Генерация видео и аудио.**

Вопрос 24: Автокодировщики (Autoencoders): идея, кодировщик, декодировщик, области применения.

Автокодировщики (Autoencoders): Это тип нейронных сетей, используемых для обучения без учителя, основной целью которых является эффективное сжатие данных и последующее их восстановление. Они учатся создавать компактное представление (кодировку) входных данных.

Идея: Автокодировщик состоит из двух основных частей:

- **Кодировщик (Encoder):** Принимает входные данные и преобразует их в более компактное, низкоразмерное представление, называемое **скрытым пространством (latent space)**, **кодировкой (encoding)** или **бутылочным горлышком (bottleneck)**. Этот процесс можно рассматривать как сжатие информации.
- **Декодировщик (Decoder):** Принимает кодировку из скрытого пространства и пытается восстановить исходные входные данные. Этот процесс можно рассматривать как распаковку информации.

Процесс обучения: Автокодировщик обучается путем минимизации **ошибки реконструкции** — разницы между исходными входными данными и данными, восстановленными декодировщиком. Цель — научиться создавать такую кодировку, которая позволяет максимально точно восстановить исходные данные.

Архитектура: Обычно Кодировщик и Декодировщик представляют собой многослойные перцептроны или сверточные нейронные сети, в зависимости от типа входных данных.

Области применения:

- **Снижение размерности:** Скрытое пространство автокодировщика является низкоразмерным представлением данных, которое можно использовать для визуализации или как вход для других моделей машинного обучения.
- **Удаление шума (Denoising Autoencoders):** Обучение автокодировщика на зашумленных данных для восстановления чистых данных. Модель учится игнорировать шум.
- **Обнаружение аномалий (Anomaly Detection):** Автокодировщик хорошо восстанавливает «нормальные» данные. Если он плохо восстанавливает какой-либо объект, это может указывать на аномалию.
- **Генерация данных (Variational Autoencoders, VAE):** Более продвинутые автокодировщики, такие как VAE, могут использоваться для генерации новых данных, похожих на обучающие.
- **Сжатие данных:** Использование кодировки для эффективного хранения данных.

Вопрос 25: Байесовские методы классификации: наивный Байес, области применения.

Байесовские методы классификации: Это класс алгоритмов классификации, основанных на **теореме Байеса**. Они используют вероятностный подход для предсказания класса объекта, вычисляя вероятность того, что объект принадлежит к определенному классу, учитывая его признаки.

Теорема Байеса:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Где:

- $P(A|B)$ — апостериорная вероятность: вероятность события А при условии, что событие В произошло.
- $P(B|A)$ — вероятность события В при условии, что событие А произошло.
- $P(A)$ — априорная вероятность: вероятность события А до учета события В.
- $P(B)$ — полная вероятность события В.

В контексте классификации:

$$P(\text{Класс}|\text{Признаки}) = \frac{P(\text{Признаки}|\text{Класс})P(\text{Класс})}{P(\text{Признаки})}$$

Цель — найти класс, для которого $P(\text{Класс}|\text{Признаки})$ максимально.

Наивный Байес (Naïve Bayes): Это простой, но эффективный алгоритм классификации, основанный на теореме Байеса. Он называется «наивным» из-за сильного (и часто неверного) предположения о **независимости признаков** при условии принадлежности к классу. То есть, он предполагает, что каждый признак независим от других признаков, если известен класс объекта.

Как работает:

1. **Вычисление априорных вероятностей классов:** Определяется вероятность каждого класса в обучающем наборе (например, доля спам-писем).
2. **Вычисление условных вероятностей признаков:** Для каждого признака и каждого класса вычисляется вероятность появления этого признака при условии принадлежности к данному классу (например, вероятность слова «акция» в спам-письме).
3. **Предсказание:** Для нового объекта вычисляется апостериорная вероятность для каждого класса, используя теорему Байеса и предположение о независимости признаков. Объект относится к тому классу, для которого эта вероятность максимальна.

Формула для Наивного Байеса:

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

Где y — класс, x_i — признаки.

Области применения:

- **Классификация текстов:** спам-фильтры, категоризация документов, анализ тональности.
- **Медицинская диагностика:** предсказание заболеваний на основе симптомов.
- **Рекомендательные системы:** (хотя сейчас чаще используются более сложные методы).
- **Прогнозирование погоды.**

Преимущества:

- Прост в реализации и быстр в обучении.
- Хорошо работает на больших наборах данных.
- Эффективен для текстовых данных.

Недостатки:

- «Наивное» предположение о независимости признаков редко выполняется в реальном мире, что может снижать точность.
- Чувствителен к нулевым вероятностям (если признак не встречался в обучающем наборе для данного класса, его вероятность будет 0, что обнулит все произведение). Это решается с помощью сглаживания (например, сглаживание Лапласа).

Вопрос 26: Методы обработки несбалансированных данных (Oversampling, Undersampling, SMOTE).

Несбалансированные данные (Imbalanced Data): Это ситуация в задачах классификации, когда количество примеров одного класса (миноритарного) значительно меньше, чем количество примеров другого класса (мажоритарного). Например, обнаружение мошенничества (очень мало мошеннических транзакций) или диагностика редких заболеваний.

Проблема: Модели, обученные на несбалансированных данных, склонны игнорировать миноритарный класс, так как оптимизация общей точности (ассурагу) будет достигаться за счет правильного предсказания мажоритарного класса. Это приводит к плохой производительности на миноритарном классе, который часто является более важным.

Методы обработки несбалансированных данных:

- **Oversampling (Перевыборка):** Увеличение количества примеров миноритарного класса, чтобы сбалансировать распределение классов.

Идея: Создать больше копий существующих примеров миноритарного класса или сгенерировать новые синтетические примеры.

Простые методы Oversampling:

- **Случайная перевыборка (Random Oversampling):** Простое дублирование случайных примеров миноритарного класса.
Недостатки: Может привести к переобучению, так как модель будет видеть одни и те же примеры много раз.
- **Undersampling (Недовыборка):** Уменьшение количества примеров мажоритарного класса, чтобы сбалансировать распределение классов.

Идея: Удалить часть примеров мажоритарного класса.

Простые методы Undersampling:

- **Случайная недовыборка (Random Undersampling):** Случайное удаление примеров мажоритарного класса. **Недостатки:** Может привести к потере важной информации, содержащейся в удаленных примерах мажоритарного класса.
- **SMOTE (Synthetic Minority Over-sampling Technique):** Один из наиболее популярных и эффективных методов перевыборки, который генерирует **синтетические** примеры миноритарного класса, а не просто дублирует существующие.

Как работает:

1. Для каждого примера миноритарного класса выбираются его k ближайших соседей (из того же миноритарного класса).
2. Затем выбирается один из этих k соседей.
3. Новый синтетический пример создается путем интерполяции между исходным примером и выбранным соседом (то есть, берется точка на отрезке, соединяющем эти два примера).

Преимущества SMOTE:

- Создает новые, разнообразные примеры, что снижает риск переобучения по сравнению с простым дублированием.
- Помогает модели лучше обобщать на миноритарный класс.

Другие подходы:

- **Изменение функции потерь:** Присвоение большего веса ошибкам на миноритарном классе.

- **Использование ансамблевых методов:** Некоторые ансамблевые методы (например, AdaBoost) по своей природе хорошо справляются с несбалансированными данными.

Вопрос 27: Интерпретируемость моделей: SHAP, LIME.

Интерпретируемость моделей (Model Interpretability): Это способность человека понимать, почему модель машинного обучения приняла то или иное решение. В условиях растущей сложности моделей (например, глубоких нейронных сетей) интерпретируемость становится критически важной, особенно в таких областях, как медицина, финансы или юриспруденция, где необходимо объяснять решения.

SHAP (SHapley Additive exPlanations): Это метод объяснения предсказаний любой модели машинного обучения. Он основан на концепции **значений Шепли** из теории кооперативных игр. SHAP вычисляет, насколько каждый признак (или группа признаков) вносит вклад в предсказание модели для конкретного экземпляра данных.

Идея: Значения Шепли распределяют «выигрыш» (в данном случае — предсказание модели) между «игроками» (признаками) справедливо, учитывая все возможные комбинации признаков. SHAP предоставляет единую меру важности признака, которая имеет следующие свойства:

- **Локальная точность:** Объяснение должно быть точным для конкретного предсказания.
- **Отсутствие признака:** Если признак отсутствует, его вклад должен быть нулевым.
- **Согласованность:** Если модель изменяется так, что признак имеет большее влияние, его значение SHAP должно увеличиться.

Преимущества SHAP:

- **Модель-агностик:** Может использоваться с любой моделью машинного обучения.
- **Локальные и глобальные объяснения:** Может объяснять как отдельные предсказания, так и общую важность признаков для всей модели.
- **Теоретически обоснован:** Основан на строгой математической теории.

LIME (Local Interpretable Model-agnostic Explanations): Это еще один популярный метод для объяснения предсказаний любой модели машинного обучения. LIME фокусируется на создании **локальных, интерпретируемых объяснений** для отдельных предсказаний.

Идея: Для объяснения конкретного предсказания LIME:

1. **Генерирует возмущенные версии** исходного экземпляра данных (например, для изображения — путем изменения пикселей, для текста — путем удаления слов).
2. **Получает предсказания** основной (сложной) модели для этих возмущенных экземпляров.
3. **Обучает простую, интерпретируемую модель** (например, линейную регрессию или дерево решений) на этих возмущенных данных и их предсказаниях, взвешивая их по близости к исходному экземпляру.
4. **Использует простую модель** для объяснения предсказания сложной модели для исходного экземпляра. Эта простая модель является локальной аппроксимацией поведения сложной модели в окрестности данного экземпляра.

Преимущества LIME:

- **Модель-агностик:** Работает с любой моделью.
- **Локальная интерпретируемость:** Предоставляет объяснения, которые легко понять человеку для конкретного предсказания.
- **Визуализация:** Особенно хорошо подходит для визуализации объяснений для изображений и текста.

Отличия SHAP и LIME:

- SHAP основан на теории игр и дает более «справедливое» распределение вкладов признаков. LIME строит локальную аппроксимацию.
- SHAP часто более вычислительно затратен, чем LIME, особенно для большого количества признаков.
- Оба метода являются мощными инструментами для повышения доверия к моделям машинного обучения.

Вопрос 28: Принципы MLOps: CI/CD для ML, мониторинг моделей, управление версиями данных и моделей.

MLOps (Machine Learning Operations): Это набор практик, который объединяет разработку машинного обучения (ML), операции (Ops) и инженерию данных. Цель MLOps — автоматизировать и стандартизировать жизненный цикл моделей машинного обучения, от разработки и обучения до развертывания, мониторинга и обслуживания в производственной среде. MLOps является расширением DevOps для систем ML.

Основные принципы MLOps:

- **CI/CD для ML (Continuous Integration/Continuous Delivery for ML):** Расширение традиционных концепций CI/CD для включения специфических аспектов машинного обучения.
 - **Непрерывная интеграция (CI):** Включает автоматическое тестирование кода, данных и моделей. При каждом изменении кода или данных, пайплайн CI должен автоматически переобучать модель, тестировать ее производительность и проверять на наличие регрессий.
 - **Непрерывная доставка (CD):** Автоматизация процесса развертывания обученных моделей в производственную среду. Это может включать развертывание модели как API-сервиса, встраивание в приложение или обновление существующей модели.
 - **Непрерывное обучение (CT - Continuous Training):** Автоматическое переобучение моделей в производственной среде при появлении новых данных или изменении распределения данных (дрейф данных).
- **Мониторинг моделей (Model Monitoring):** Критически важный аспект MLOps, который включает постоянное отслеживание производительности развернутых моделей в реальном времени. Цель — выявлять проблемы, такие как снижение точности, дрейф данных или смещение предсказаний, чтобы можно было своевременно принять меры.

Что мониторится:

- **Производительность модели:** Метрики (accuracy, precision, recall, F1, RMSE, MAE) на реальных данных.

- **Дрейф данных (Data Drift):** Изменение распределения входных данных со временем, что может сделать модель устаревшей.
- **Дрейф концепции (Concept Drift):** Изменение связи между входными признаками и целевой переменной.
- **Отклонение предсказаний (Prediction Drift):** Изменение распределения выходных предсказаний модели.
- **Ресурсы:** Использование CPU, GPU, памяти, задержка ответов.

При обнаружении проблем мониторинг должен запускать оповещения и, возможно, автоматическое переобучение или откат к предыдущей версии модели.

- **Управление версиями данных и моделей (Data and Model Versioning):** Обеспечение возможности отслеживать и воспроизводить любую версию данных и моделей, используемых в жизненном цикле ML. Это критически важно для воспроизводимости экспериментов, отладки и аудита.
 - **Версионирование данных:** Отслеживание изменений в наборах данных, используемых для обучения и тестирования. Это включает не только сами данные, но и метаданные (источник, дата, преобразования). Инструменты: DVC (Data Version Control).
 - **Версионирование моделей:** Сохранение различных версий обученных моделей, включая их архитектуру, гиперпараметры, метрики производительности и данные, на которых они были обучены. Это позволяет легко откатываться к предыдущим версиям или сравнивать их.

Управление версиями данных и моделей позволяет обеспечить прозрачность, воспроизводимость и надежность систем машинного обучения.

Вопрос 29: Облачные платформы для ML (AWS SageMaker, Google Cloud AI Platform, Azure Machine Learning).

Облачные платформы для ML: Это интегрированные сервисы, предоставляемые облачными провайдерами (Amazon Web Services, Google Cloud, Microsoft Azure), которые предлагают полный набор инструментов и инфраструктуры для разработки, обучения, развертывания и управления

моделями машинного обучения. Они значительно упрощают и ускоряют процесс создания ML-решений, предоставляя масштабируемые ресурсы и готовые сервисы.

Преимущества облачных платформ для ML:

- **Масштабируемость:** Легкое масштабирование вычислительных ресурсов (CPU, GPU) по мере необходимости.
- **Управляемые сервисы:** Готовые к использованию инструменты для всех этапов жизненного цикла ML, снижающие операционную нагрузку.
- **Интеграция:** Бесшовная интеграция с другими облачными сервисами (хранилища данных, базы данных, аналитика).
- **Сотрудничество:** Упрощение совместной работы команд.
- **Безопасность:** Встроенные механизмы безопасности и соответствия стандартам.

Основные облачные платформы:

- **AWS SageMaker (Amazon Web Services SageMaker):** Комплексный сервис от Amazon, который предоставляет разработчикам и специалистам по данным возможность быстро создавать, обучать и развертывать модели машинного обучения.

Ключевые возможности:

- **SageMaker Studio:** Единая IDE для всего жизненного цикла ML.
- **Notebook Instances:** Управляемые Jupyter Notebooks.
- **Алгоритмы и фреймворки:** Встроенные алгоритмы и поддержка популярных фреймворков (TensorFlow, PyTorch, Scikit-learn).
- **Обучение:** Масштабируемое распределенное обучение, автоматическое тюнинг гиперпараметров.
- **Развертывание:** Развертывание моделей в виде API-конечных точек с автоматическим масштабированием.
- **Мониторинг:** Мониторинг производительности моделей и дрейфа данных.
- **Data Labeling:** Инструменты для разметки данных.

- **Google Cloud AI Platform (сейчас часть Vertex AI):** Предложение Google Cloud для машинного обучения, которое объединяет различные сервисы ML в единую платформу. Vertex AI — это унифицированная платформа для создания, развертывания и масштабирования моделей ML.

Ключевые возможности Vertex AI:

- **Workbench:** Управляемые Jupyter Notebooks.
 - **Training:** Обучение моделей с использованием пользовательских контейнеров или встроенных алгоритмов.
 - **Prediction:** Развертывание моделей для онлайн- и пакетного прогнозирования.
 - **Managed Datasets:** Управление наборами данных и их версионирование.
 - **Feature Store:** Централизованное хранилище признаков для повторного использования.
 - **ML Metadata:** Отслеживание метаданных экспериментов.
 - **Explainable AI:** Инструменты для интерпретируемости моделей.
 - **AutoML:** Автоматическое создание моделей без написания кода.
- **Azure Machine Learning (Microsoft Azure Machine Learning):** Платформа от Microsoft Azure, предназначенная для ускорения создания и развертывания моделей ML. Она предлагает инструменты как для специалистов по данным, так и для разработчиков.

Ключевые возможности:

- **Azure Machine Learning Studio:** Веб-интерфейс для управления проектами ML.
- **Notebooks:** Интегрированные Jupyter Notebooks.
- **Automated ML:** Автоматический выбор алгоритмов и тюнинг гиперпараметров.
- **Designer:** Drag-and-drop интерфейс для создания ML-пайплайнов без кода.
- **MLflow Integration:** Поддержка MLflow для отслеживания экспериментов.

- **Model Deployment:** Развертывание моделей в контейнерах (Docker) на различных вычислительных ресурсах.
- **Responsible ML:** Инструменты для обеспечения справедливости, интерпретируемости и конфиденциальности моделей.

Все эти платформы предоставляют мощные инструменты для всего жизненного цикла ML, позволяя командам сосредоточиться на создании ценности, а не на управлении инфраструктурой.

Вопрос 30: Большие данные: основные характеристики (3V/5V), технологии (Hadoop, Spark), MapReduce.

Большие данные (Big Data): Это термин, описывающий чрезвычайно большие и сложные наборы данных, которые невозможно эффективно обрабатывать и анализировать с помощью традиционных методов и инструментов обработки данных. Большие данные характеризуются объемом, скоростью и разнообразием, что требует специализированных технологий для их хранения, обработки и анализа.

Основные характеристики Больших данных (3V/5V):

- **Volume (Объем):** Огромное количество данных, измеряемое в терабайтах, петабайтах, эксабайтах и более. Это может быть объем данных, генерируемых социальными сетями, IoT-устройствами, транзакциями и т.д.
- **Velocity (Скорость):** Скорость, с которой данные генерируются, собираются и должны быть обработаны. Это может быть потоковая передача данных в реальном времени (например, данные с датчиков, клики на веб-сайтах, финансовые транзакции).
- **Variety (Разнообразие):** Разнообразие форматов и типов данных. Это могут быть структурированные данные (базы данных), полуструктурированные (XML, JSON), неструктурированные (текст, изображения, видео, аудио). Разнообразие усложняет хранение и анализ.
- **Veracity (Достоверность/Качество):** Качество и достоверность данных. Большие данные часто содержат шум, неточности, неполные или противоречивые сведения. Важно уметь оценивать и управлять качеством данных.

- **Value (Ценность):** Способность извлекать ценную информацию и знания из больших данных для принятия решений. Без извлечения ценности, большие данные бесполезны.

Технологии для Больших данных:

- **Hadoop (Apache Hadoop):** Это фреймворк с открытым исходным кодом, предназначенный для распределенного хранения и обработки очень больших наборов данных на кластерах обычных компьютеров. Hadoop состоит из нескольких ключевых компонентов:
 - **HDFS (Hadoop Distributed File System):** Распределенная файловая система, которая хранит данные на множестве машин, обеспечивая высокую доступность и отказоустойчивость.
 - **MapReduce:** Модель программирования для параллельной обработки больших данных на кластере (подробнее ниже).
 - **YARN (Yet Another Resource Negotiator):** Система управления ресурсами кластера, которая планирует и управляет задачами.

Особенности Hadoop: Надежность, масштабируемость, отказоустойчивость, но может быть медленным для итерационных задач из-за записи промежуточных результатов на диск.

- **Spark (Apache Spark):** Это мощный движок для обработки больших данных, который значительно быстрее Hadoop MapReduce, особенно для итерационных алгоритмов и интерактивного анализа. Spark может работать как поверх HDFS, так и с другими системами хранения.

Ключевые особенности Spark:

- **Обработка в памяти:** Значительно ускоряет операции, храня данные в оперативной памяти между этапами обработки.
- **Унифицированный движок:** Поддерживает различные типы рабочих нагрузок: пакетную обработку, потоковую обработку, SQL-запросы, машинное обучение (MLlib) и графовые вычисления (GraphX).
- **API на разных языках:** Поддерживает Scala, Java, Python (PySpark), R.
- **Resilient Distributed Datasets (RDD):** Основная абстракция Spark, представляющая собой неизменяемую, распределенную коллекцию элементов, которую можно обрабатывать параллельно.

MapReduce: Это модель программирования и соответствующая реализация для обработки больших наборов данных с использованием параллельного, распределенного алгоритма на кластере. Она состоит из двух основных фаз:

- **Map (Отображение):**

- Входные данные разбиваются на более мелкие части.
- Функция `map` обрабатывает каждую часть независимо, преобразуя ее в набор пар «ключ-значение».
- Например, для подсчета слов в тексте, `map` может принимать строку текста и выдавать пары (слово, 1) для каждого слова.

- **Shuffle and Sort (Перемешивание и Сортировка):**

- Промежуточный этап, где все пары «ключ-значение» с одинаковыми ключами группируются вместе.
- Например, все пары (слово, 1) для одного и того же слова будут собраны вместе.

- **Reduce (Свертка):**

- Функция `Reduce` принимает сгруппированные пары «ключ-значение» и агрегирует их.
- Например, для подсчета слов, `Reduce` будет принимать (слово, [1, 1, 1, ...]) и суммировать единицы, выдавая (слово, количество).

Преимущества MapReduce:

- **Параллелизм:** Позволяет обрабатывать огромные объемы данных параллельно на множестве машин.
- **Отказоустойчивость:** Система автоматически обрабатывает свои узлы.
- **Масштабируемость:** Легко масштабируется путем добавления новых машин в кластер.

Недостатки MapReduce:

- **Медленность для итерационных задач:** Из-за записи промежуточных результатов на диск между фазами.

- **Сложность программирования:** Требуется специфический подход к решению задач.

Spark был разработан как более гибкая и быстрая альтернатива MapReduce, особенно для задач, требующих многократных итераций над данными.